

An Open-Source Dual-Camera Edge AI Runtime for Vision-Guided Drone Autonomy on an MCU-EdgeTPU Platform

Author Name ^{1,*}

¹ Affiliation, City, Country; email@example.com

* Correspondence: email@example.com

Highlights

What are the main findings?

- Our MCU-EdgeTPU board extends the original Coral Micro board from which the design started with dual-camera sensing, onboard IMU support, MicroPython control, and both EdgeTPU-backed and MCU-side inference paths for compact autonomous-drone experiments.
- The implementation indicates that sustained onboard perception on this class of hardware depends strongly on RTOS scheduling, memory placement, and hardware-assisted image preparation rather than on accelerator execution alone.

What are the implications of the main findings?

- Lightweight UAV platforms can support richer onboard perception when sensing, memory movement, inference placement, and telemetry are co-designed as one embedded system.
- A compact open-source platform can serve as a reproducible testbed for studying onboard inference, runtime adaptation, and bounded high-level tool use in drone applications.

Abstract

This paper presents our MCU-EdgeTPU board, an open-source inference platform derived from the original Coral Micro board from which the design started and extended with two 5 Mpx cameras, an onboard IMU, and a single MicroPython runtime. The platform is intended to make onboard inference accessible from scripts, not only by exposing accelerator access, but by integrating sensing, model execution, communication, and runtime utilities into a single embedded environment. The runtime supports both EdgeTPU-backed execution for quantized models compiled for accelerator use and CPU-side TensorFlow Lite Micro execution for auxiliary workloads that remain on the MCU. The platform targets two complementary UAV workflows: Bitcraze Crazyflie for agile indoor experimentation and PX4/MAVLink-oriented autopilot integration for more transferable field deployment. At the perception level, the runtime provides a dual-camera, multi-model execution path with camera switching, per-camera geometry, lightweight tracking, motion compensation, and ground-aware spatial reasoning. A central contribution is the systems-level engineering of the visual path from sensor capture to EdgeTPU input, including DMA-backed acquisition, hardware-assisted image preparation, memory placement across internal RAM and SDRAM, and pipelined FreeRTOS scheduling that overlaps capture, preprocessing, and inference. The implementation therefore treats throughput, buffer ownership, and accelerator feeding as first-class design constraints rather than secondary optimization details. Beyond the runtime itself, the work is accompanied by an open synthetic dataset and a custom YOLO model intended for low-altitude drone navigation in inhabited environments. Taken

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Drones* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

together, these elements define an open embedded platform for onboard inference experiments on a compact airborne system: integrated sensing, accelerator-backed and MCU-side inference paths, and a substantial set of embedded-ML functions available directly from MicroPython. Together with the accompanying synthetic aerial dataset and custom YOLO model, the work extends the original Coral Micro board from which the design started into a more complete environment for onboard inference, adaptive embedded ML, and lightweight autonomous-drone research.

Keywords: embedded AI; EdgeTPU; MicroPython; drone autonomy; MAVLink; Crazyflie; dual camera; FreeRTOS; embedded inference platform

1. Introduction

Autonomous visual navigation for small drones sits at the intersection of several conflicting constraints: low mass, low power, real-time perception, onboard decision making, and enough software flexibility to support rapid iteration. In practice, many platforms solve only part of this problem. Very small aerial vehicles are easy to fly indoors and safe to test in tight environments, but they often lack the compute and memory budget required by modern object detection pipelines. At the opposite end, larger embedded AI platforms such as Coral Dev Board or NVIDIA Jetson-class systems provide significantly more resources, but they also bring higher current consumption, larger batteries, more weight, and a form factor that is difficult to justify for lightweight micro-UAV missions.

This work starts from the original Coral Micro board from which the design started and treats it not as a fixed product, but as a compact open platform that can be extended through systems engineering. The goal is not to add isolated features, but to assemble our MCU-EdgeTPU board as an embedded-AI stack with improved sensing, more predictable runtime behavior, and a clearer control and telemetry model.

The starting point of this work was the need for a platform that remains physically lightweight and energy-efficient, while still exploiting as much as possible the inference capability of the EdgeTPU and the real-time performance of the MCU. The original minimalist EdgeTPU-based board configuration was attractive from the perspective of size and power, but its standard low-resolution camera workflow was not sufficient for the class of missions targeted here. In particular, a 320x240 sensing regime is too restrictive when the goal is to support richer object detection, stricter scene understanding, and higher-confidence flight decisions in inhabited or cluttered spaces. This became even more limiting when considering modern YOLO-style detectors, which benefit from larger spatial inputs and from a sensor pipeline capable of preserving more detail before resizing and quantization.

The objective is therefore to define a different point in the design space: a board-level and software-level architecture that preserves the low-weight, low-power character of minimalist embedded AI hardware, but extends it toward an open inference platform for autonomous-drone operation. The platform couples an MCU and an EdgeTPU with two 5 Mpx cameras and an onboard IMU, and exposes sensing, inference, communication, and control through MicroPython. High-level mission logic remains scriptable, while timing-critical operations stay in the RTOS and accelerator path. The runtime is also no longer limited to a single inference route: it includes both EdgeTPU-backed execution for quantized models compiled for accelerator execution and CPU-side TensorFlow Lite Micro execution, together with a growing family of embedded-ML utilities intended to be used directly from scripts.

The work explicitly targets two flight platforms. The first is Bitcraze Crazyflie, used as an indoor experimental platform because it is agile, lightweight, easy to maneuver, and well suited for repeated safety-conscious testing in the laboratory. The second is a PX4-based autopilot stack accessed through MAVLink, representing the path toward a more professional and transferable deployment model. The intention is to provide a common high-level autonomy layer across both targets, so that mission scripts, perception logic, and communication flows can be developed once and adapted across indoor research flights and more operational MAVLink-centered scenarios. This broader PX4-facing integration matters because the runtime is designed not only to emit textual or backend-oriented telemetry, but also to provide structured obstacle information that can be consumed directly by autopilot-side collision-prevention logic.

From a vision-systems perspective, the work is driven by the observation that model accuracy alone is not enough. Once YOLO-class detectors are pushed toward larger inputs, for example 640-class or even workflows derived from native 1280-wide camera imagery, the dominant issues become memory layout, transfer scheduling, tensor preparation cost, and buffer ownership between the camera subsystem, the MCU, and the EdgeTPU. For this reason, the contribution is as much about systems engineering as it is about AI. The implementation analyzes the full visual path from image acquisition at the sensor, through DMA-backed capture into RAM, through hardware pixel conversion and scaling, and finally into the EdgeTPU input memory. These stages are pipelined in parallel under the RTOS so that camera capture and image preparation can overlap with inference, thereby maximizing frame rate while minimizing idle time and unnecessary copies.

An important architectural extension is the addition of a dual-camera subsystem. Instead of treating perception as a single fixed viewpoint problem, the platform supports two 5 Mpx sensors that may be mounted in parallel or perpendicular configurations. This enables distinct operational modes: forward-looking navigation together with nadir inspection, redundant views for indoor localization, or viewpoint switching according to the phase of the mission. The camera-switching mechanism is designed as a first-class runtime concept, not as an afterthought, and is supported by explicit per-camera geometry, frame sequencing, and clean post-switch acquisition logic. Together with IMU-aware tracking and ground-plane projection, this creates the basis for drone-centric perception that is aware of mounting geometry and flight context rather than only raw image content.

The work also extends beyond the runtime itself. In addition to dynamic model loading and multi-model switching, the project assumes the availability of a custom YOLO detector trained on an open synthetic dataset built for low-altitude drone navigation in inhabited spaces. The dataset contains views captured at nadir and at approximately 45 degrees, and is intended as a reusable pre-training or adaptation resource for future systems that must obey stricter navigation and avoidance logic in populated environments, including scenarios informed by European drone categories such as A0-A3 and A2. This dataset-and-model layer complements the runtime by providing a path from open training assets to experimentally reproducible, EdgeTPU-oriented inference models.

The present work should therefore be read as an integration effort. Embedded Python, edge inference, object detection, MAVLink communication, visual tracking, and lightweight learning primitives are all known in isolation. The difficulty is making them coexist on a highly constrained airborne platform without losing throughput, determinism, or usability. The practical bottlenecks addressed here include RAM placement across internal memory and SDRAM, large tensor staging, RTOS synchronization, camera-switching latency, conversion of tracked detections into obstacle maps, and the need to sustain high throughput without giving up the low-power character of the platform.

The main contributions of the work can be summarized as follows:

1. An open-source MicroPython runtime for a lightweight EdgeTPU-enabled MCU platform intended for high-level autonomous drone control, with both EdgeTPU and CPU-side TensorFlow Lite Micro inference paths.
2. A dual-target flight-control concept spanning Bitcraze Crazyflie for indoor experimentation and PX4/MAVLink for professional autopilot integration.
3. A dual-camera 5 Mpx sensing architecture with onboard IMU support, runtime camera switching, per-camera geometry, and viewpoint-aware operation.
4. A real-time perception pipeline that studies and optimizes the end-to-end path from image sensor to MCU RAM to EdgeTPU memory, using parallel RTOS execution and double-buffered staging to maximize frame rate.
5. A memory-aware deployment strategy for YOLO-class detectors and other larger vision models on an MCU-constrained platform.
6. A multi-model workflow with dynamic model loading, context-dependent switching, tracking, projection to the ground plane, obstacle reporting to MAVLink autopilots, and telemetry integration.
7. An extended embedded-ML runtime surface that includes on-device learning, clustering, anomaly detection, sequence modeling, reinforcement learning, and lightweight mapping primitives as integrated MicroPython-accessible components alongside the vision stack.
8. An open synthetic aerial dataset and a custom YOLO model intended as a starting point for regulation-aware drone navigation in inhabited environments.

The rest of the paper is organized as follows. The next section reviews related work in autonomous nano-UAV navigation, embedded deep learning, YOLO-based detection, MicroPython-enabled embedded control, and accelerator-centric edge AI. The following sections present the hardware and software architecture, the real-time memory and processing pipeline, the communication and flight-control abstractions, and the experimental perspective enabled by the platform.

2. Related Work

2.1. Autonomous visual navigation for nano-UAVs

An important reference point for this work is the line of research that demonstrates fully onboard visual autonomy on extremely small flying robots. A particularly relevant example is *An Open Source and Open Hardware Deep Learning-powered Visual Navigation Engine for Autonomous Nano-UAVs*, which shows that a nano-UAV can execute a closed-loop visual navigation pipeline onboard under severe power and mass constraints. That work is important not only because it proves feasibility, but because it frames the design problem correctly: perception, inference, control, and hardware architecture cannot be treated independently when the air vehicle is very small. The platform developed here follows this systems-level viewpoint, but shifts the focus from a custom navigation engine toward a more general-purpose embedded AI runtime that combines high-level scripting, object detection, multi-view sensing, and compatibility with both experimental and professional drone-control ecosystems.

The broader nano-UAV literature also distinguishes between systems that are truly autonomous onboard and systems that depend on offboard computing, external localization infrastructure, or heavy companion computers. This distinction remains essential. For indoor research, offboard support is useful during development, but it does not solve the core embedded autonomy problem. Our MCU-EdgeTPU board therefore sits within the class of systems that aim to keep perception and high-level decision making onboard, even when this requires substantial engineering effort in memory organization, RTOS scheduling, and sensor integration.

2.2. *Embedded AI accelerators and systolic-array inference*

The use of an EdgeTPU-class accelerator places this work within the wider context of edge inference on specialized matrix-processing hardware. Systolic-array accelerators changed the practical landscape of embedded AI because they made it possible to execute neural inference at high throughput and low energy relative to CPU-only or even general GPU-centric approaches. In this broader context, the TPU family established a strong reference model for efficient tensor computation through structured dataflow and regular multiply-accumulate pipelines. This work is not a study of accelerator microarchitecture by itself, but it depends critically on the availability of this kind of accelerator model and on the practical challenge of feeding it efficiently from a constrained MCU environment.

This distinction matters. On larger boards, accelerator throughput is often discussed independently from sensor transfer and host memory layout, because the host has enough resources to hide inefficiencies. On small MCU-based systems, however, the effective performance of the accelerator is inseparable from the cost of moving images from the camera to RAM, rearranging tensors, and scheduling inference without starving the rest of the application. In that sense, this work contributes to the embedded AI literature by focusing on the host-side industrialization problem around the accelerator, not only on the accelerator itself.

2.3. *YOLO-family object detectors and compact detection models*

The object detection component of this work is aligned with the large body of literature around single-stage detectors and especially the YOLO family. YOLO-style architectures are attractive in airborne robotics because they offer a practical balance between accuracy, latency, and deployment simplicity. They are also natural candidates for quantized execution and edge deployment. At the same time, deploying YOLO-class models on resource-constrained embedded platforms introduces a very different set of concerns from those seen on desktop GPUs: input resolution becomes a memory-placement problem, post-processing cost becomes part of the frame budget, and intermediate copies become unacceptable at scale.

Within this design space, compact variants such as YOLO26-style models are especially relevant because they seek to preserve detection usefulness while reducing computational and memory pressure. This makes them well matched to embedded AI pipelines in which the native camera imagery may be relatively large, but the runtime still needs to fit within tight SRAM and SDRAM limits. Our platform sits precisely at this interface between compact model design and deployment-aware systems engineering. Rather than treating the detector as an isolated black box, it studies how detector choice interacts with camera resolution, quantization, memory layout, and real-time scheduling.

2.4. *MicroPython and high-level programmability on embedded systems*

MicroPython has long been valued in embedded systems for shortening the iteration cycle between algorithm design, hardware interaction, and system testing. In robotics, this is particularly attractive because state machines, mission logic, communication rules, and experimental procedures change quickly, often faster than low-level firmware can be safely recompiled and reflashed. However, the majority of MicroPython deployments emphasize convenience over hard real-time performance and are usually not coupled to a high-throughput edge-AI pipeline.

The present platform uses MicroPython differently: not as a replacement for the real-time system, but as a high-level orchestration layer placed above it. Time-critical operations such as frame capture, scaling, quantization, inference, and buffering remain implemented in compiled code under the RTOS, while Python exposes them as programmable mission

primitives. This approach is relevant to previous work on embedded scripting because it shows how an interpreted environment can be integrated into an accelerator-centric, real-time drone application without turning the interpreter into the performance bottleneck.

2.5. Lightweight embedded learning, sequence modeling, and SLAM primitives

The newer `sentai` namespaces also connect the platform to a broader line of work in statistical learning and robotics. The `kmeans` and `pca` modules draw on widely used clustering and dimensionality-reduction methods that remain attractive on microcontroller-class systems because they are compact, interpretable, and easy to integrate into larger pipelines [1,2]. The `anomaly` module fits the same pattern: it relies on lightweight statistical monitoring techniques that are practical when memory and compute budgets do not permit heavier end-to-end methods [3,4].

The sequence-oriented modules map just as cleanly onto established post-1990 literature. Dynamic time warping remains a practical baseline for elastic matching of short temporal patterns [5], while hidden Markov models provide a compact probabilistic framework for temporal-state inference [6]. The reinforcement-learning surface draws from tabular Q-learning, bandit strategies such as UCB and Thompson sampling, and later neural approximations exemplified by DQN [7–10]. The `slam` namespace likewise sits within the modern SLAM literature on probabilistic mapping and localization [11,12]. The contribution here is not novelty in these algorithms themselves, but their exposure as compact components inside the same embedded runtime that already handles sensing, inference, and flight interoperability.

2.6. Vision pipelines for drone perception and viewpoint-aware sensing

Most embedded drone-vision pipelines still assume a single active camera and a relatively static geometry. This is reasonable in many systems, but it limits mission flexibility. Drone operation frequently benefits from multiple sensing attitudes: forward views for obstacle awareness and navigation, nadir views for inspection and target localization, and oblique views that balance situational context with ground visibility. This work extends that discussion by treating dual-camera operation and runtime camera switching as core design elements. The system therefore aligns with research on viewpoint-aware aerial perception, but approaches the problem from the perspective of deployment on a compact embedded AI stack rather than from the perspective of large-scale perception benchmarks alone.

This also connects with work on projection-aware tracking and sensor fusion. Once IMU measurements, camera geometry, and ground-plane assumptions are available, detections can be represented not only in image coordinates but also in more operationally meaningful spatial coordinates. The implementation builds on this idea by combining tracking, IMU-aware motion compensation, per-camera geometry, and projection to the ground plane so that perception can be consumed more directly by flight logic and telemetry systems. In tracking terms, the design sits between classical SORT [13] and later ByteTrack-style association [14]: it preserves a simple Kalman-based online core and the recovery of lower-confidence detections, but replaces heavier desktop-oriented assumptions with MCU-suitable mechanisms such as static memory, compact histogram cues, and IMU-based motion compensation.

2.7. Open datasets, synthetic data, and regulation-aware navigation

Synthetic data has become increasingly important in aerial robotics because collecting and annotating large real-world datasets from drones is expensive, repetitive, and often constrained by safety or regulation. For applications involving inhabited spaces, this difficulty is even greater: the scenes are operationally sensitive, the failure modes are

costly, and the data should ideally cover multiple viewing angles and flight contexts. An open synthetic dataset that explicitly contains nadir and oblique perspectives is therefore consistent with current trends in simulation-assisted pre-training and domain adaptation.

The relevance of this direction becomes stronger when drone navigation is discussed not only as a perception problem, but also as a compliance problem. European operation categories such as A0-A3 and A2 motivate systems that are more conservative, more aware of surrounding objects and persons, and easier to validate under explicit operational constraints. While regulation-aware autonomy is still an emerging topic, it provides an important framing device for this work: the detector and dataset are not meant only to improve benchmark accuracy, but to support future high-level navigation policies that must operate under stricter safety and legal constraints.

2.8. Positioning of this work

Relative to prior work, our MCU-EdgeTPU board occupies a middle ground that is not addressed well by any single existing direction. It is lighter and more power-conscious than larger embedded AI computers, more programmable than fixed-function onboard vision stacks, more perception-rich than minimal MCU-only controllers, and more systems-oriented than many standalone model papers. Its contribution is therefore not a new detector alone, not a new autopilot alone, and not a new board alone, but the integration of these concerns into an embedded autonomy platform for small drones.

Indicative references to include in the final bibliography are the following:

1. *An Open Source and Open Hardware Deep Learning-powered Visual Navigation Engine for Autonomous Nano-UAVs.*
2. Core YOLO-family papers and the compact YOLO26 line relevant to efficient embedded detection.
3. Foundational work on TPU and systolic-array-based neural accelerators.
4. MicroPython project and related work on embedded scripting for robotics.
5. Prior work on aerial perception, synthetic datasets for UAV vision, and regulation-aware autonomous flight.

3. Materials and Methods

This chapter describes the implementation of the platform as a research system positioned at the intersection of embedded AI acceleration, real-time systems, and autonomous drone control. The central thesis of the implementation is that an EdgeTPU-class accelerator is simultaneously low-power and computationally powerful enough to justify a much tighter integration with an MCU than is common in larger companion-computer architectures. Rather than treating the accelerator as a peripheral attached to a heavy host, the design treats the MCU and EdgeTPU as a compact co-processing pair: the MCU handles sensing, scheduling, memory orchestration, communication, and high-level control, while the EdgeTPU is driven as the main execution substrate for neural inference.

An equally important framing point is that the implementation starts from the original Coral Micro board from which the design started and deliberately pushes it toward industrialization. In other words, the work is not only about extending that open-source base with additional capabilities, but about transforming it into our MCU-EdgeTPU board, a system that exhibits more production-oriented behavior: higher sustained throughput, better control of memory and latency, richer sensor integration, clearer runtime abstractions, and a software structure that can support repeated deployment rather than isolated demonstrations.

The implementation presented in this work differs substantially from the initial fork baseline by transforming a board-oriented demo environment into a programmable

autonomous-flight runtime. The main engineering question is not only whether inference can run onboard, but how to maximize accelerator utilization, sustain real-time responsiveness, and preserve a lightweight and low-power hardware profile suitable for micro-UAV operation. This leads to a design in which memory placement, transfer scheduling, camera integration, scripting boundaries, and telemetry abstractions become first-order implementation concerns. In this sense, the chapter documents a move from open-source proof-of-potential to an embedded system engineered with production-performance objectives in mind.

The chapter is organized into the following subchapters:

1. [implementation_01_system_architecture.md](#), which explains the hardware-software co-design and the motivation for pairing an MCU with an EdgeTPU on a lightweight drone-oriented board.
2. [implementation_02_runtime_and_memory.md](#), which analyzes the RTOS integration, linker-level memory layout, staging buffers, and the host-side logic required to keep the accelerator fed efficiently.
3. [implementation_03_vision_pipeline.md](#), which details the dual-camera subsystem, the real-time image path, tracking, projection, and the deployment of YOLO-class detectors.
4. [implementation_04_flight_and_telemetry.md](#), which presents the dual-target control strategy spanning Crazyflie and PX4/MAVLink, together with telemetry and backend integration.
5. [implementation_05_programming_model.md](#), which studies the MicroPython layer and uses the platform help system as evidence of how the embedded capabilities are exposed as a coherent experimental interface.

Taken together, these sections argue that the implementation is best understood as an exercise in practical embedded-AI industrialization: the work does not merely deploy neural inference on a small board, but systematically resolves the hardware and software bottlenecks that arise when such inference must coexist with flight control, camera switching, runtime programmability, and constrained memory resources. The overall contribution is therefore to show how an open-source platform can be matured toward a more production-capable embedded autonomy stack without abandoning its accessibility, compactness, or low-power design philosophy.

3.1. System Architecture and Design Rationale

Figure 1 summarizes the layered view of the platform, from sensing hardware and RTOS orchestration to EdgeTPU inference and the MicroPython experimentation surface.

System Architecture and Design Rationale

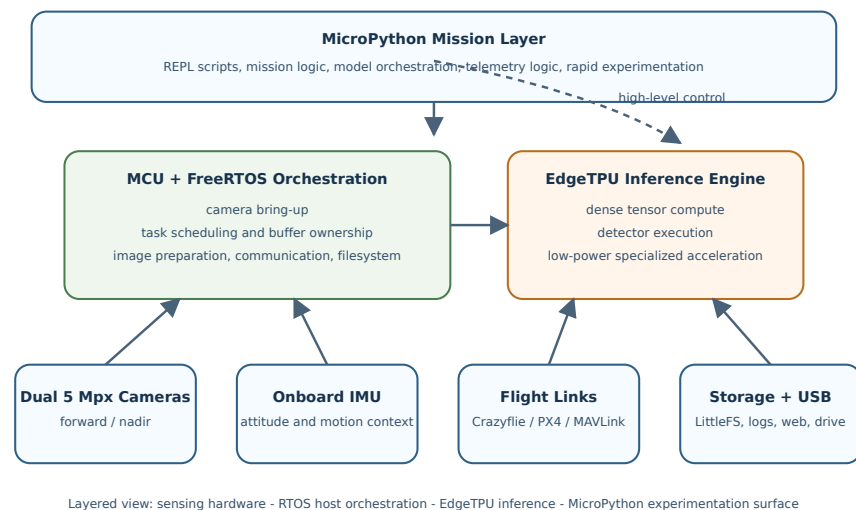


Figure 1. Block-level view of the platform architecture, highlighting the split between sensing hardware, MCU orchestration, EdgeTPU inference, and the script-oriented MicroPython layer.

The implementation starts from a strong architectural hypothesis: for lightweight autonomous drones, the most useful computing arrangement is not a large companion computer with a small flight controller attached to it, but a tightly coupled MCU-plus-accelerator system in which the accelerator performs the dense tensor computation and the MCU orchestrates everything else. This view is consistent with broader trends in domain-specific architectures for neural networks, especially the systolic-array lineage popularized by TPU-style accelerators, where high arithmetic density and low energy per operation make specialized inference hardware much more attractive than general-purpose processing for continuously running perception tasks.

In this work, the EdgeTPU is treated as the primary neural execution engine and the MCU as the real-time systems controller that makes this engine usable in practice. The MCU is responsible for sensor bring-up, camera switching, image scaling and quantization setup, task scheduling under FreeRTOS, telemetry transport, filesystem access, and the high-level control hooks exposed to the drone. The choice of a real-time operating system, rather than a Linux-based software stack, is deliberate. For this class of airborne system, FreeRTOS offers lower software overhead, tighter control over scheduling and memory, faster boot behavior, simpler timing analysis, and a substantially smaller energy and storage footprint than a full operating-system distribution. This division of responsibility is central to the implementation strategy. It avoids the weight and current draw associated with larger embedded AI platforms while still enabling onboard deep-learning inference for navigation and scene understanding.

This design choice is particularly important in the drone context. Platforms such as Coral Dev Board or Jetson-class systems are powerful, but their mass, thermal requirements, battery burden, and system complexity make them difficult to justify for small indoor drones or for compact airframes where endurance and safety matter. Their software model is also usually tied to a Linux-class environment, which is convenient for development but less attractive when deterministic timing, reduced boot latency, and strict power discipline are required onboard a lightweight UAV. By contrast, a minimalist EdgeTPU-enabled board can remain lightweight and power-conscious, yet still sustain meaningful onboard perception if the software stack is built around the accelerator and an RTOS-controlled MCU rather than around the comfort assumptions of a desktop-like host. The implementation

therefore argues that the EdgeTPU is not merely a convenient inference add-on, but a sufficiently strong low-power compute primitive to be integrated beside virtually any capable MCU, provided that memory and data movement are engineered with care.

The concrete system extends the minimal board profile with two 5 Mpx cameras and an onboard IMU. This is a deliberate departure from lower-resolution single-camera operating modes commonly found in embedded vision examples. The intent is to preserve more visual detail at the sensing stage, even when the final detector input is smaller, because high-resolution acquisition creates more room for strategic cropping, scaling, model switching, and camera-role specialization. In practical terms, this matters for YOLO-class detectors, which are sensitive to input detail and often benefit from larger or better-preserved spatial structure before quantization.

The two-camera design also reflects a broader systems argument. A single fixed camera is often enough for a benchmark, but not always enough for a drone mission. Indoor navigation, target localization, regulatory awareness, and scene understanding may require forward-looking and nadir-looking views, or rapid switching between complementary perspectives. The implementation supports this by treating the dual-camera subsystem as part of the platform architecture, not as an optional accessory. Camera identity, mount geometry, and active-view selection are propagated all the way to the tracking and projection layers.

At the software level, the architecture is also shaped by programmability. Rather than exposing the platform only through C++ firmware entry points, the system embeds a full MicroPython runtime and binds the major capabilities of the board into a structured `sentai` module. This follows the intuition that autonomous flight research evolves too quickly for every experimental change to require a full firmware rewrite. High-level mission logic, model orchestration, telemetry handling, and flight behaviors are therefore scripted, while the timing-critical path remains implemented in compiled code.

This leads to a deliberate split between two programming models. At the lower level, the firmware is task-oriented in C++: camera acquisition, preprocessing, communication, tracking, and control services are organized as explicit FreeRTOS tasks, queues, semaphores, and singleton runtime components with well-defined ownership over buffers and peripherals. At the upper level, the platform becomes script-oriented in MicroPython: the user composes these services through short scripts, REPL commands, and boot-time logic without needing to manage the underlying task graph directly. The purpose of this split is not only convenience, but architectural clarity. C++ remains responsible for determinism, concurrency, and performance-critical resource control, while MicroPython remains responsible for orchestration, experimentation, and rapid reconfiguration of behavior.

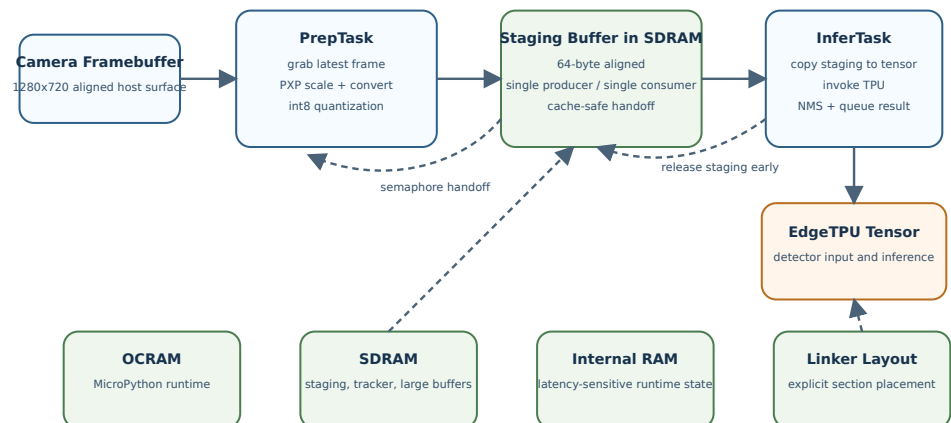
In this way, the implementation combines the rapid iteration benefits associated with embedded scripting with the deterministic execution required by real-time perception.

The resulting system should thus be understood as a co-designed stack with four tightly linked layers: hardware sensing and capture, real-time host orchestration on the MCU, neural inference on the EdgeTPU, and a high-level control and experimentation surface in MicroPython. This layered organization is what makes the platform useful as a research artifact rather than merely as a single-purpose demo.

3.2. RTOS Integration, Memory Layout, and Accelerator Feeding

Figure 2 shows the runtime pipeline used to keep the accelerator fed, with explicit staging, task handoff, and memory placement.

RTOS Integration, Memory Layout, and Accelerator Feeding



Pipeline view: aligned capture - PrepTask - SDRAM staging - InferTask - EdgeTPU, with explicit memory placement and cache discipline

Figure 2. Block diagram of the RTOS and memory pipeline, showing aligned capture buffers, PrepTask, SDRAM staging, InferTask, and the EdgeTPU tensor path.

One of the main implementation claims of this work is that achieving strong onboard vision performance on a compact MCU-plus-EdgeTPU platform depends less on raw inference speed alone and more on how efficiently the host prepares, places, and transfers data. This is particularly visible in the code added relative to the initial fork, where the runtime evolves from a board example into a carefully staged real-time system.

The core execution environment is FreeRTOS. Rather than running perception as a single sequential loop, the implementation decomposes the workload into cooperating tasks with explicit ownership boundaries. The most relevant example is the continuous detection pipeline, where the code in `detection_task.cc` organizes the system around a preparation task and an inference task. The preparation task acquires the latest frame, scales and converts it through the hardware pixel pipeline, applies quantization when needed, and writes the result into a staging buffer. The inference task copies the prepared staging contents into the EdgeTPU input tensor, releases the staging buffer immediately, and then executes inference and post-processing while preparation for the next frame can already proceed. This design is not simply an optimization; it is the mechanism through which the implementation turns limited memory and compute resources into sustained throughput.

An important low-level detail is that the runtime does not feed the detector directly from the raw camera bus format. The active camera configuration captures native 1280x720 frames into 64-byte-aligned framebuffers with a 4-byte-per-pixel host representation. In practice, the software treats these buffers as XRGB8888-like surfaces because this layout is easier to move through CSI DMA, cache maintenance, and the RT1176 pixel pipeline than a tightly packed 3-byte RGB tensor. This is precisely where the PXP block becomes essential rather than optional: it converts the aligned capture surface into RGB888 output at the detector input size, eliminating expensive per-pixel software shuffles on the Cortex-M7.

This organization also clarifies the broader software split introduced in the architecture chapter. The lower implementation layer is task-oriented in C++, meaning that concurrency is expressed directly through FreeRTOS tasks, semaphores, queues, and explicit buffer handoff between producer and consumer stages. By contrast, the upper user layer remains script-oriented in MicroPython, where these same services are invoked as high-level operations without exposing the user to the internal synchronization structure. The runtime and

memory subsystem is precisely where this separation becomes most meaningful: scripts can request model loading, camera capture, or continuous detection, but the correctness and performance of those operations depend on the task-oriented C++ layer maintaining deterministic ownership over memory, timing, and accelerator access.

The use of double-buffered staging is especially important because it enforces a clean ownership discipline between producer and consumer stages. The staging buffer is written by the preparation side and read by the inference side under semaphore-controlled handoff, while the TPU tensor is written only when the staging contents are ready. This minimizes race conditions, reduces needless synchronization complexity, and preserves the ability to overlap work. In a larger system with abundant memory, one could compensate for poor orchestration by brute force. In the present implementation, orchestration is the only realistic way to maintain high frame rate while preserving the low-power character of the platform.

The same section of code also shows why byte alignment becomes a first-order systems concern. The staging area in SDRAM is explicitly aligned to cache boundaries, while the PXP conversion path performs cache clean-and-invalidate operations before DMA writes and invalidation after completion so that stale boundary lines do not overwrite fresh image data. This is needed because the TFLite arena is not naturally aligned to the same cache-line granularity as the DMA engine. Without that discipline, a seemingly minor mismatch between 16-byte tensor allocation and 32-byte cache-line behavior would produce intermittent corruption exactly at the stage where the detector input is assembled.

The memory layout reflects the same philosophy. The linker script introduces a deliberate partitioning across internal memory, OCRAM, heap, SDRAM, and a dedicated region for camera-related traffic. MicroPython is placed explicitly in OCRAM rather than in the most latency-sensitive memory regions, while large buffers, tracker state, and staging areas are pushed into SDRAM where capacity matters more than absolute access latency. This is visible both in the custom linker script and in the pervasive use of explicit section placement such as `.sdram_bss`. The implementation thus accepts that not all memory is equal and uses the linker as part of the optimization strategy rather than as a passive build artifact.

This memory strategy also explains a design choice that would otherwise look surprising from a purely model-centric viewpoint: the system first tolerates a larger, alignment-friendly framebuffer representation and only later collapses it into the compact RGB tensor expected by the model. In other words, the runtime pays for structure before it pays for compactness. That tradeoff is justified because aligned DMA and deterministic hardware conversion are cheaper on this platform than repeatedly manipulating tightly packed pixels in software.

This issue becomes more acute once YOLO-class detectors are considered. Larger detector inputs significantly increase the pressure on staging buffers, copied tensors, post-processing data, and runtime metadata. In a conventional desktop pipeline this is almost invisible. On an MCU platform it becomes decisive. The code therefore embodies a memory-rearrangement strategy aimed at fitting the MicroPython runtime, the real-time tasks, the tracker structures, and the camera-to-TPU pipeline into a workable whole while minimizing inefficient copies. This is one of the places where the implementation differs most strongly from a conceptual prototype: the problem is no longer simply to run a model, but to run it without collapsing the rest of the system.

The same logic informs the build system. The custom CMake configuration aggregates a full embedded MicroPython build, nanopb-generated protocol code, additional audio and communication components, and the custom runtime sources into a single executable with a purpose-built linker script. This makes the firmware itself part of the experimental

method. The implementation is not a loose collection of demos, but a custom-built runtime whose compilation model, memory map, and dependencies are aligned with the targeted deployment conditions.

From a research perspective, this subsection supports a broader claim: on compact embedded AI platforms, the practical efficiency of a specialized accelerator depends on the host system's ability to keep the accelerator supplied with valid input at the right pace. The implementation therefore treats memory organization and RTOS scheduling as intrinsic parts of accelerator utilization rather than as secondary engineering details.

3.3. Dual-Camera Vision Pipeline and Perception Stack

Figure 3 illustrates how the two cameras, preprocessing path, detector, tracker, and geometry-aware projection stages interact inside the perception stack.

Dual-Camera Vision Pipeline and Perception Stack

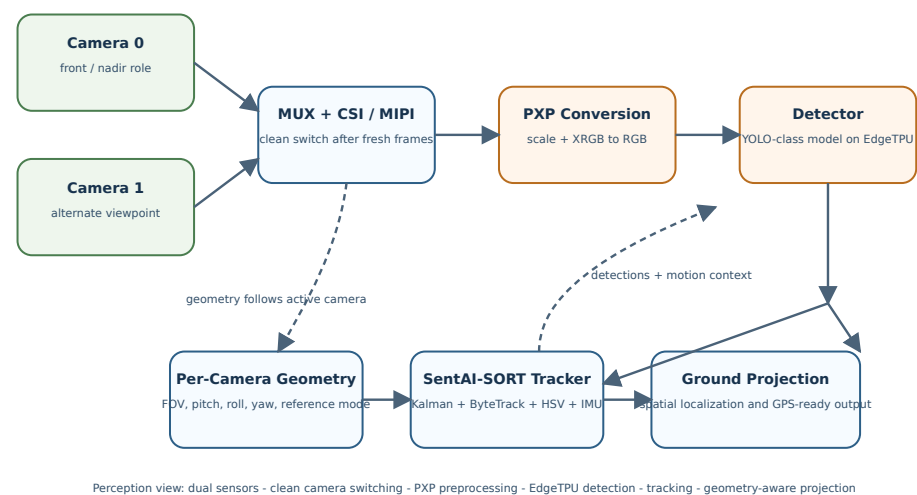


Figure 3. Dual-camera perception pipeline with camera switching, PXP preprocessing, EdgeTPU detection, tracking, and ground-aware projection.

The perception implementation is built around the idea that high-quality airborne vision is constrained as much by sensing geometry and data movement as by the neural model itself. For this reason, the codebase extends the original single-board camera model into a dual-camera system with explicit support for runtime switching, per-camera geometry, and high-resolution acquisition.

At the capture level, the cameras are integrated through the processor camera subsystem and the MIPI CSI-2 path. This is an important architectural choice because it prevents the image path from becoming dominated by the kind of software-managed peripheral transfer overhead that would be unacceptable for sustained real-time inference. The camera stack is designed so that CSI and MIPI remain active even during switching, which reduces disruption and helps preserve throughput. In practical terms, the runtime exposes `sentai.camera.switch(id)` while internally snapshotting the frame sequence and waiting until enough fresh frames have arrived from the newly selected sensor to guarantee a clean image. This is a small but important example of the thesis that real embedded autonomy depends on disciplined treatment of edge cases that benchmark-oriented prototypes often ignore.

The OV5640 integration itself is more substantial than a generic sensor bring-up. Register control is handled over SCCB through the board I2C interfaces, including model-ID

verification, reset and power-down sequencing, per-camera mirror and rotation adjustments, and sensor-specific initialization on each side of the camera multiplexer. On the video side, the datasheet-level interface still matters to the architecture: the sensor family supports compact formats such as 8-bit RGB565, but the host capture path on the RT1176 is engineered around aligned 32-bit framebuffers and DMA-friendly pitches. As a result, the implementation deliberately separates transport format concerns from model-input format concerns instead of assuming that the detector can consume the capture representation directly.

The implementation also recognizes that switching cameras is not useful unless geometry travels with the switch. The perception pipeline therefore maintains per-camera configuration for field of view, mount pitch, mount roll, mount yaw, and ground-reference mode. This allows the same runtime to support parallel cameras, perpendicular camera placement, nadir sensing, or oblique views without hard-coding assumptions about the sensor pose. In aerial robotics this is essential because the operational meaning of a detection depends on how the camera is mounted, not only on where the pixels appear.

The neural perception path itself is intentionally deployment-aware. Frames are captured at the sensor side, then scaled and converted through the hardware pixel pipeline before being prepared for the model input tensor. This means the system can begin from richer camera imagery, including workflows derived from native 1280-wide sensing, while still feeding smaller quantized tensors to the detector. Such a path is crucial for compact YOLO-family models, which often live in tension between the need for more spatial detail and the strict memory limits of MCU deployment.

One fine but important engineering point is that the PXP stage is not used only for resizing. The camera receiver exposes raw host buffers whose layout is convenient for aligned capture, cache maintenance, and bus transfer, but not for direct detector input. The runtime therefore uses PXP to transform XRGB-style capture surfaces into packed RGB output with the exact detector dimensions, and then quantizes in place if the TPU model expects int8 input. This avoids repeated software byte shuffling, avoids ambiguities caused by aligned scanline storage, and turns pixel-format conversion into a deterministic hardware stage of the pipeline.

Above the detector, the implementation adds a custom tracking layer referred to in code as SentAI-SORT. The design borrows the logic of modern tracking-by-detection systems while adapting it to the realities of a Cortex-M7-class host. The tracker combines an 8-state Kalman formulation, a ByteTrack-style two-stage association process, and compact HSV histogram features. Instead of relying on expensive feature-based motion compensation, it uses IMU-informed compensation derived from camera motion. This substitution is not only computationally cheaper, but also better aligned with the sensing resources already present on the platform.

It is useful to state explicitly how this tracker differs from established baselines. Relative to the original SORT formulation [13], the implementation is less minimalist in association: it does not rely only on predicted box overlap, but recovers lower-confidence detections through a second association stage in the spirit of ByteTrack [14], and it adds a lightweight appearance cue through a compact HSV-grid histogram. Relative to ByteTrack itself, however, the goal here is not leaderboard-oriented MOT performance on a workstation-class host. The implementation removes assumptions that are natural in desktop tracking pipelines but costly on an MCU, such as dependence on richer appearance embeddings, larger temporary buffers, or host-side post-processing flexibility. In practice, SentAI-SORT is best understood as a bounded embedded tracker that keeps the constant-velocity Kalman core and the useful low-confidence recovery logic, but compresses the rest of the design to fit a static-memory, real-time inference loop.

This redesign was necessary for systems reasons rather than for novelty in tracking alone. On this platform, the tracker executes synchronously after detector output, with all state kept in statically allocated memory and with a total footprint small enough to coexist with camera buffers, tensor staging, MicroPython, and the rest of the runtime. The appearance term is therefore reduced to a small histogram representation instead of a learned re-identification branch, and camera motion compensation is driven by the onboard IMU rather than by feature extraction and image-to-image homography estimation. These choices trade some generality for a much tighter computational envelope, but they also exploit information that is naturally available on an airborne embedded system. Just as importantly, the tracker is not an isolated MOT module: it is integrated with per-camera geometry, ground-plane projection, and event export, so that tracked objects can be consumed directly by flight logic, telemetry, or higher-level navigation code rather than remaining only image-space trajectories.

The tracker is further extended with ground-plane projection and optional GPS-referenced localization. Once altitude, heading, IMU attitude, and camera geometry are known, detections can be mapped beyond image coordinates into a spatial frame meaningful for the drone and for downstream consumers. This is particularly important when the runtime is used not only to detect objects, but also to generate position-aware telemetry or to feed higher-level navigation policies.

The implementation therefore supports a wider claim than simple onboard detection. It demonstrates that a small embedded AI platform can host a complete airborne perception chain: dual high-resolution sensing, viewpoint switching, quantized detection, lightweight tracking, motion compensation, and spatial projection. In combination with model switching, this also creates the conditions for multi-model operation in which different detectors or visual policies can be applied according to mission phase, camera viewpoint, or scene constraints.

Finally, the perception stack is complemented conceptually by the custom YOLO model and the synthetic open dataset discussed elsewhere in the paper. Although those assets are external to the code implementation, they are consistent with the implementation logic described here: the runtime is engineered so that compact detection models trained for nadir and oblique drone views can be loaded dynamically and used as part of a practical onboard autonomy workflow.

3.4. Flight Control, Telemetry, and External Integration

Figure 4 summarizes the dual-target control model, in which one runtime connects local perception to both Crazyflie and PX4/MAVLink-centered workflows.

Flight Control, Telemetry, and External Integration

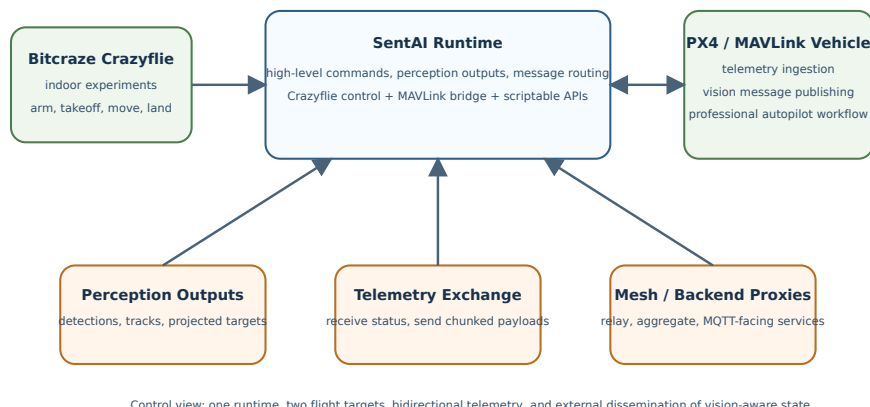


Figure 4. Control and telemetry architecture spanning Crazyflie experimentation, MAVLink interoperability, and external backend dissemination.

The implementation does not stop at perception. A central design goal is to make perception actionable in flight-relevant terms, and this is why the runtime introduces two distinct but complementary control paths: one centered on Bitcraze Crazyflie for indoor experimentation and one centered on PX4-compatible communication through MAVLink for a more conventional autopilot ecosystem.

The Crazyflie path is valuable because it gives the research platform an accessible indoor drone target that is lightweight, inexpensive, and safe to iterate on in constrained spaces. In implementation terms, the runtime exposes high-level flight functions for arming, takeoff, landing, attitude control, hover-like motion, waypoint movement, and basic link validation. The significance of this layer is less in the underlying protocol details and more in the abstraction boundary it creates: the experimental logic written in MicroPython can command a real flying platform without descending into firmware-specific implementation code for every new experiment.

The MAVLink path plays a different role. It positions the same perception and scripting environment toward a standard autopilot workflow, enabling connection to ground-control software and to a PX4-style vehicle architecture. In the implementation, the MAVLink bridge is built around the official message library and a dedicated receive task, with queue-based delivery of decoded telemetry into the runtime. This allows the platform to consume motion, position, and status telemetry while also publishing onboard detections and system messages back to the wider vehicle ecosystem. The result is a direct technical bridge between a compact onboard perception unit and a more conventional autopilot stack.

This bridge is not limited to a single control primitive. The runtime exposes both transmission and reception semantics at the scripting layer, including reception of autopilot telemetry, chunked transmission of structured vision payloads that would not fit cleanly into a single small embedded message, and direct generation of PX4-compatible OBSTACLE_DISTANCE messages for collision-prevention workflows. In practical terms, this means the same runtime can send high-level detection events through compact serialized vision messages, or it can transform tracker-ground projections and explicit point sets into the radial obstacle maps expected by a professional autopilot. That detail matters because it

shows the design is meant for sustained exchange between perception and flight software rather than for one-shot demo messages.

The obstacle-reporting path is particularly important because it closes the loop between onboard perception and flight-safety behavior. Instead of treating detections only as information for logging or remote display, the runtime can reinterpret tracked objects as navigational constraints in the vehicle frame and publish them through MAVLink as a dense angular distance map. This is a more operational use of the perception stack than simple text telemetry: the embedded vision system becomes capable of feeding an autopilot-side collision-prevention mechanism with geometry derived from its own tracker or from higher-level mapping outputs such as SLAM landmarks [11,12].

This dual-target approach is important conceptually. Many embedded AI demonstrators either remain tied to a single experimental robot or aim immediately at a professional stack without a low-risk iteration platform. Here, the implementation supports both modes. Crazyflie serves as a rapid prototyping platform for indoor validation, while MAVLink provides compatibility with broader drone workflows. From a research standpoint, this strengthens the portability of the proposed autonomy layer.

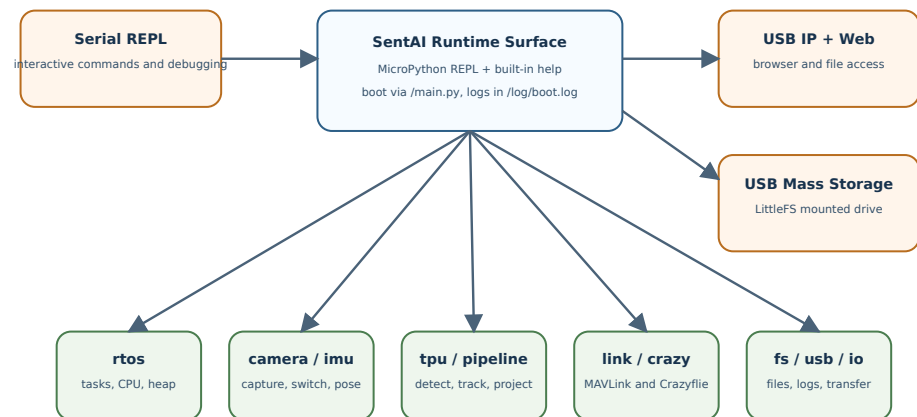
The communication stack extends further through mesh and backend services. The firmware side includes a compact binary vision message protocol and bridges for wireless dissemination, while the repository also adds backend proxy services that relay, aggregate, and expose telemetry through ground-side interfaces such as MQTT. This is significant because it shows that the implementation is intended to operate as part of a distributed autonomy pipeline rather than as an isolated board demo. Detections are not merely computed; they are structured, transported, and made available to external systems.

In implementation terms, the control and communication layers provide the operational dimension of embedded perception. A detector running locally is useful only to a point. A detector that can feed a Crazyflie experiment, inform a MAVLink autopilot context, emit autopilot-consumable obstacle maps, publish compact messages to external services, and remain scriptable from a high-level runtime is more appropriate for systems research than a detector evaluated only in isolation. This is the level at which the runtime begins to resemble an onboard autonomy substrate rather than a perception benchmark harness.

3.5. Programming Model and User-Facing Runtime Surface

Figure 5 provides a compact view of the user-facing runtime model, including the three USB access paths and the main groups of the `sentai` namespace.

Programming Model and User-Facing Runtime Surface



User-facing view: three access modes on USB, one coherent sentai namespace, and script-driven control over sensing, inference, communication, and logs

Figure 5. Programming model view showing REPL-centered access, USB service modes, boot artifacts, and the main runtime modules exposed through the `sentai` namespace.

An implementation of this kind must be evaluated not only by its internal efficiency but also by how coherently it exposes its capabilities to the user. This is especially important on MCU-based platforms, where debugging is often more difficult than on Linux-class systems because the developer has less runtime introspection, fewer interactive tools, and a much narrower margin for trial-and-error when something goes wrong during boot or during a real-time task interaction. For this reason, the platform does not expose a single debugging path, but a small set of complementary connection modes over USB that make the board significantly easier to inspect and operate in practice.

The first mode is an interactive MicroPython REPL available over the serial interface, which allows a researcher or operator to connect directly to the board, inspect the runtime state, execute commands live, load scripts, and request contextual help from within the system itself. This is a consequential design choice: the platform is not only flashed and executed as a fixed firmware image, but can also be interrogated and steered interactively during development and testing. In practice, the REPL becomes the primary entry point for experimentation, rapid diagnosis, and iterative mission design.

The second mode is USB networking over IP, through which the firmware exposes an onboard web server and file-browser interface. This gives the board a lightweight service surface that can be reached directly from the host, making it possible to inspect and transfer artifacts without depending exclusively on the serial console. The third mode is USB mass storage, in which the LittleFS user partition is exposed as a mounted drive so that images, scripts, and models can be uploaded or downloaded directly from the host machine. Taken together, these three USB-facing paths, namely serial REPL, IP connectivity with web access, and mounted LittleFS storage, form a practical answer to the usual debugging and deployment friction of MCU-based systems.

The built-in help interface available from the REPL makes this programming model discoverable without forcing the user to leave the target system. This matters because the runtime is relatively rich: it does not expose a single detector call or a single flight command, but a full namespace of platform primitives intended to support sensing, inference, telemetry, storage, and high-level control from one embedded environment. The same environment also supports script-driven startup through `/main.py`, which is automatically executed at boot before the interactive session begins. This gives the platform an important

dual character: it can behave as an interactive research instrument during development, but it can also behave as a self-starting embedded application once the desired mission logic has been written.

This startup behavior is complemented by a boot logging mechanism implemented directly in the runtime. During initialization, console and driver output are captured into `/log/boot.log`, while the previous boot log is rotated to `/log/boot_old.log`. This is an important practical feature for embedded debugging, because many failures on an MCU platform occur before a user can attach interactively to the REPL. By preserving the initialization trace on LittleFS, the system allows the user to inspect the outcome of FreeRTOS bring-up, early peripheral initialization, and startup script execution even after the board has already moved past the failing point.

The `sentai` namespace now organizes the platform into a broader set of functional modules than the original runtime surface, including `io`, `rtos`, `tpu`, `tfl`, `fs`, `camera`, `imu`, `mic`, `usb`, `uart`, `mesh`, `link`, `crazy`, `sleep`, `pipeline`, `aifes`, `kmeans`, `pca`, `anomaly`, `dtw`, `hmm`, `r1`, and `slam`. This structure is significant because it reveals the intended user abstraction: the operator is not expected to manipulate a collection of unrelated firmware endpoints, but to work with a coherent runtime in which device functions are grouped by task and by experimental purpose. The platform can query RTOS state, inspect heap usage, load EdgeTPU or CPU-side TensorFlow Lite models, capture or switch cameras, read inertial measurements, record audio, enter low-power idle states, exchange data over USB, UART, Meshtastic, or MAVLink, start a continuous detector, track objects, build simple online models, or command a drone, all through a single embedded language environment.

This helps explain why MicroPython is a central implementation choice rather than a peripheral convenience. The interpreter does not replace the lower-level real-time system; instead, it defines a stable experimentation layer above it. The help text demonstrates this duality clearly. On one hand, users access simple operations such as `sentai.camera.to_tensor()`, `sentai.tpu.invoke()`, or `sentai.tfl.invoke()` depending on whether inference is mapped to the EdgeTPU or kept on the MCU. On the other hand, the same interface reaches into advanced platform features such as tracking events, per-camera geometry, ground-plane projection, obstacle reporting, and MAVLink message exchange. The implementation thereby condenses a complex firmware system into a form that is operable from concise Python scripts.

Several modules are especially important for understanding the research value of the platform. The `rtos` module exposes runtime observability primitives such as task inspection, CPU statistics, and heap information, which are essential when the system is tuned for sustained real-time throughput. The `tpu` module provides the basic EdgeTPU inference control plane: model loading, tensor access, invocation, quantization metadata, and detector-oriented post-processing. The newer `tfl` module extends this surface with CPU-side TensorFlow Lite Micro inference for models that are not compiled for the EdgeTPU, which means the runtime can now host both accelerator-backed and pure-MCU inference paths under a common scripting model. The `camera` module complements these inference interfaces by managing capture, JPEG export, transfer to the tensor path, runtime camera switching, and frame sequencing. The `imu` and `mic` modules supply the inertial and audio measurements needed by viewpoint-aware tracking, sound-triggered wake-up, and non-visual experiments. The `fs`, `usb`, and `uart` modules support a practical workflow in which models, scripts, captured outputs, and debugging artifacts can be transferred to and from the device without rebuilding the firmware for every iteration.

The additional machine-learning namespaces are also relevant because they extend the runtime beyond a detector-and-flight stack into a broader embedded inference and experimentation environment. The `aifes` module exposes on-device neural-network train-

ing and inference through the AIFES library [15]. The `kmeans`, `pca`, and `anomaly` modules add compact methods for clustering, dimensionality reduction, and anomaly scoring that are well suited to MCU-class deployment because they impose limited model overheads [1–4]. The `dtw` and `hmm` modules extend the runtime toward sequence modeling for gesture, audio, and other short temporal patterns [5,6], while the `r1` module adds adaptive control mechanisms spanning tabular Q-learning, bandits, and a compact DQN-style multilayer perceptron [7–10]. Finally, the `slam` module exposes a detection-driven EKF-SLAM interface in which object detections can be promoted from transient image measurements into a persistent two-dimensional map [11,12]. Together, these additions extend the role of the runtime: it is not only a way to load and invoke models compiled for the accelerator, but also a compact collection of integrated ML primitives that can be composed directly from MicroPython for perception, adaptation, and state-estimation experiments.

The help surface also shows that these capabilities were designed to be exercised directly from the target rather than only through host-side tooling. Typical examples include immediate tensor feeding from the camera, live detector startup, autopilot connection, or direct Crazyflie flight commands, for example:

```
sentai.camera.to_tensor()
sentai.tpu.invoke()
print(sentai.tpu.detect())
```

```
sentai.link.init(57600)
sentai.pipeline.start(0.3, 0.45, 50, True)
while True:
    msg = sentai.link.receive()
    if msg:
        print(msg)
```

```
sentai.crazy.init()
sentai.crazy.arm()
sentai.crazy.fly(0.3, 3000)
sentai.crazy.fly_stop()
```

At the control and autonomy layer, `mesh`, `link`, `crazy`, `sleep`, and `pipeline` form the most distinctive part of the namespace. The `mesh` module exposes Meshtastic-compatible communication as a first-class runtime capability for low-bandwidth distributed sensing and message dissemination. The `link` module does the same for MAVLink communication, allowing the board to exchange telemetry and structured messages with an autopilot or a ground-control system. The `crazy` module provides an analogous control surface for the indoor Crazyflie target, exposing high-level flight functions directly to Python. The `sleep` module complements these interfaces with an explicit low-power idle entry point that can wake on microphone energy or tap events, which is important for battery-constrained edge deployments. The `pipeline` module unifies the continuous detector, tracker, event stream, per-camera geometry, and pose-aware projection logic into a single interface. Together, these modules show that the platform is meant to be used as a scriptable inference-and-autonomy layer, where models, sensors, telemetry, and embedded ML primitives can be combined interactively from MicroPython.

The `pipeline` module is especially illustrative. According to the documented runtime surface, it exposes continuous detection, statistics, tracker snapshots, event streams, configurable association parameters, pose setting, and per-camera geometry. This is not a trivial wrapper around a detector. It is the public face of the deeper implementation

presented in the previous subsections: the parallel camera-to-TPU pipeline, the tracking-by-detection logic, and the geometry-aware projection model all become accessible through a minimal but expressive Python API. The same can be said for the `link` and `crazy` modules, which reveal that flight and telemetry functions are not separate applications but first-class capabilities of the runtime itself.

From a research-methodology perspective, this matters because it changes the pace and style of experimentation. A researcher can move from firmware bring-up to mission logic, data capture, tracking inspection, and communication testing without leaving the same runtime environment. This reduces iteration cost and makes the platform more suitable for exploratory autonomy research, where control policies, perception thresholds, and communication strategies change frequently.

The built-in help surface also provides indirect evidence of implementation maturity. A platform that documents precise operational behaviors, module boundaries, argument conventions, and example workflows has generally progressed beyond proof-of-concept code. Here, the fact that these capabilities can be explored directly from the serial REPL supports the claim that the implementation has been organized as a reusable experimental environment rather than as a collection of hidden firmware entry points.

The programming model therefore completes the overall argument of the chapter. The EdgeTPU provides the main accelerator-backed inference path, the MCU and RTOS provide the orchestration, the cameras, IMU, and microphone provide the sensing context, and MicroPython makes the whole system operable as a research platform. Just as importantly, the expanded `sentai` namespace shows that the implementation now spans not only perception and flight control, but also communication, adaptive learning, sequence modeling, anomaly detection, and lightweight mapping. The value of the implementation lies not only in runtime efficiency, but also in exposing that functionality through an interface that remains inspectable and extensible during autonomous-drone experiments.

4. Discussion

The preceding implementation results indicate that the main contribution of the platform lies in system integration rather than in any single algorithmic element. Its value depends on the extent to which sensing, inference, memory placement, scheduling, and communication can operate together under strict resource constraints. For this reason, a meaningful evaluation should consider not only model accuracy, but also end-to-end latency, dropped-frame behavior, camera-switch overhead, telemetry cadence, and the stability of obstacle outputs delivered to higher-level control components.

The platform also exposes a useful design space for mixed inference and lightweight adaptation. EdgeTPU-backed models, CPU-side TensorFlow Lite Micro execution, and the embedded-ML modules available through MicroPython make it possible to compare alternative placements of perception and decision components within the same runtime. This is relevant for future work on few-shot adaptation, anomaly-triggered reconfiguration, temporal modeling, and compact policy learning, where the main question is not raw model scale but how such mechanisms behave under embedded timing and memory constraints.

An additional direction concerns the use of the REPL and runtime namespaces as a bounded tool interface for an external large language model. In such a configuration, the language model would not consume raw sensor streams or issue low-level control commands. Instead, it would operate over discrete or aggregated signals produced by the device, such as detector summaries, track events, anomaly flags, obstacle bins, or mission-state variables, and would use the REPL to generate scripts or call higher-level functions. This arrangement is attractive because it preserves local execution of sensing and safety-critical logic while allowing offboard reasoning over compact symbolic state.

At the same time, the limitations of the platform remain explicit. MCU-class resources impose strong constraints on model size, buffering, concurrency, and recoverability. Any future extension involving online learning or LLM-mediated tool use would therefore need clear restrictions on admissible commands, local validation of generated actions, and strict separation between high-level reasoning and real-time control. In this sense, the platform is best viewed as a controlled environment for embedded autonomy experiments rather than as a replacement for larger companion-computer architectures.

5. Conclusions

This work presented an effort to turn a lightweight open-source embedded AI board into an open inference and autonomy platform for drones. The central result is that an EdgeTPU-class accelerator can be paired effectively with an MCU when the surrounding software stack is designed to keep sensing, memory movement, scheduling, telemetry, and programmability aligned. The goal is not to imitate a Linux-class companion computer, but to show that a much lighter board can still support meaningful onboard inference and autonomy when the system is treated as a co-design problem rather than as a loose collection of features.

The implementation extends the original Coral Micro board from which the design started with a number of capabilities required for autonomous drone research: a full MicroPython runtime, dual 5 Mpx cameras, onboard IMU integration, a parallel camera-to-EdgeTPU perception pipeline, CPU-side TensorFlow Lite Micro inference, lightweight tracking and projection, runtime camera and model switching, interactive debugging and deployment paths, obstacle-map reporting for PX4-style collision prevention, dual-target control support spanning both Bitcraze Crazyflie and PX4/MAVLink-style workflows, and an integrated set of classical ML primitives that can be invoked directly from scripts. Taken together, these additions define our MCU-EdgeTPU board as a more complete embedded inference environment.

One of the clearest conclusions is that the hardest problems in compact airborne AI systems often sit outside the neural model. Much of the implementation effort went into ensuring that images could be acquired, transferred, staged, quantized, and scheduled in real time without breaking the low-power and low-mass profile of the platform. Host-side systems engineering, especially memory placement, ownership discipline, boot-time observability, and developer-facing tooling, proved just as important as the accelerator.

The platform also contributes a broader methodological point. By combining a task-oriented C++ runtime with a script-oriented MicroPython layer, the system shows that it is possible to preserve real-time discipline while still exposing an interface suitable for rapid experimentation. This is particularly valuable in drone autonomy, where sensing configuration, communication policies, flight behaviors, and even the choice between accelerator-backed and MCU-only models are rarely fixed early in development. The ability to move repeatedly between low-level performance engineering and high-level scripting is one of the main reasons the resulting system is useful as a research artifact. The same interface now supports a wider set of embedded ML experiments, including clustering, anomaly scoring, sequence modeling, reinforcement learning, and lightweight SLAM, which further supports the view of the runtime as an environment for embedded inference workflows rather than a narrow detector wrapper.

Finally, the work suggests that lightweight open-source embedded AI boards should not be viewed only as educational or prototyping tools. With sufficient attention to integration quality, memory engineering, runtime design, and flight-stack interfaces, they can serve as technically relevant foundations for more demanding autonomous systems. The

resulting platform stands both as a concrete implementation and as an argument for taking accessible open hardware seriously as a basis for embedded autonomy research.

5.1. Future Work

Several directions follow naturally from the present implementation. A first line of future work concerns completing the symmetry between the two flight targets. Although the platform already exposes both Crazyflie-oriented and MAVLink-oriented control paths, a fuller unification of mission abstractions, telemetry semantics, and safety behaviors across both environments would strengthen the claim of a truly portable high-level autonomy layer. This includes deeper integration with PX4 control modes, more explicit offboard control patterns, richer use of onboard obstacle maps inside autopilot workflows, and a cleaner common contract between indoor experimental flights and more operational autopilot-driven deployments.

On the perception side, future work should extend the current multi-camera and multi-model logic toward tighter cross-camera continuity. The present system supports runtime camera switching, per-camera geometry, and viewpoint-aware projection, but does not yet provide full track handoff across cameras. A next step would therefore be to reproject existing track hypotheses across viewpoints and preserve track identity when a target moves from one sensor to the other. This would make the dual-camera architecture more than a switching mechanism and turn it into a more integrated multi-view perception system.

A second major direction concerns model deployment. The current work argues for compact YOLO-style detectors and open synthetic pre-training assets, but there remains significant room for improvement in model adaptation, quantization robustness, and scene-specific specialization. Future work could evaluate alternative compact detectors, mixed-model cascades, task-specific switching policies, and better domain adaptation between synthetic and real aerial imagery. It would also be valuable to formalize when workloads should remain on the EdgeTPU and when they should fall back to CPU-side TensorFlow Lite Micro execution, especially in missions that mix fast detection with lightweight auxiliary models. In particular, the regulation-aware navigation framing introduced in this work would benefit from models trained not only for general detection, but also for explicit recognition of flight-relevant constraints in inhabited spaces.

The expanded embedded-ML runtime opens a further line of work beyond classical detector deployment. The current implementation already exposes clustering, anomaly detection, sequence modeling, reinforcement learning, and lightweight SLAM primitives, but these remain building blocks rather than a mature learning framework. Future work could evaluate how these modules interact in closed-loop autonomy settings, how they should share memory and telemetry channels with the perception stack, and which of them are practically robust enough for long-duration field experiments on an MCU-class platform.

Another important direction is deeper optimization of the memory and dataflow path. The current implementation already uses task parallelism, staging buffers, and explicit SDRAM placement, but more aggressive zero-copy strategies, richer DMA-assisted transfer patterns, or compiler-assisted layout optimization may further improve throughput and energy efficiency. The same is true for the host-side interaction with the EdgeTPU: a richer study of tensor residency, batching patterns, or asynchronous execution opportunities could reveal additional performance margins on this class of embedded platform.

The developer experience can also be extended. The current system already provides a serial REPL, IP-based access with a web interface, mounted LittleFS transfer, boot logging, and boot-time execution through `main.py`. Future work could unify these into a more

formal device-management workflow with remote script deployment, structured runtime tracing, automated collection of telemetry and images, artifact export for both obstacle and vision messages, and reproducible experiment packaging. Such tooling would be especially useful if the platform is to be used by multiple researchers or integrated into longer experimental campaigns.

At the system level, the project also opens a path toward broader hardware generalization. One of the arguments made throughout this work is that the EdgeTPU is a sufficiently capable low-power inference primitive to justify close integration with an MCU. A natural next step is therefore to explore how much of the present design can be ported to other MCU families, other camera configurations, or future open-source boards with similar accelerator-centric architectures. If successful, this would strengthen the broader claim that lightweight accelerator-plus-MCU designs are a viable class of embedded autonomy platform rather than a single-board exception.

Taken together, these directions suggest that the present work is less an endpoint than a foundation. The current implementation establishes the runtime, sensing, memory, control, and experimentation abstractions needed for a compact embedded autonomy stack. Future work can now build on this base to improve viewpoint continuity, flight-platform symmetry, regulation-aware perception, deployment tooling, embedded learning workflows, and cross-platform generalization.

1. Jain, A.K. Data Clustering: 50 Years beyond K-Means. *Pattern Recognition Letters* **2010**, *31*, 651–666. <https://doi.org/10.1016/j.patrec.2009.09.011>.
2. Jolliffe, I.T.; Cadima, J. Principal Component Analysis: A Review and Recent Developments. *Philosophical Transactions of the Royal Society A* **2016**, *374*, 20150202. <https://doi.org/10.1098/rsta.2015.0202>.
3. Chandola, V.; Banerjee, A.; Kumar, V. Anomaly Detection: A Survey. *ACM Computing Surveys* **2009**, *41*, 1–58. <https://doi.org/10.1145/1541880.1541882>.
4. Pimentel, M.A.F.; Clifton, D.A.; Clifton, L.; Tarassenko, L. A Review of Novelty Detection. *Signal Processing* **2014**, *99*, 215–249. <https://doi.org/10.1016/j.sigpro.2013.12.026>.
5. Salvador, S.; Chan, P. Toward Accurate Dynamic Time Warping in Linear Time and Space. *Intelligent Data Analysis* **2007**, *11*, 561–580. <https://doi.org/10.3233/IDA-2007-11508>.
6. Bilmes, J.A. A Gentle Tutorial of the EM Algorithm and its Application to Parameter Estimation for Gaussian Mixture and Hidden Markov Models. Technical Report ICSI-TR-97-021, International Computer Science Institute, 1998.
7. Watkins, C.J.C.H.; Dayan, P. Q-Learning. *Machine Learning* **1992**, *8*, 279–292. <https://doi.org/10.1007/BF00992698>.
8. Auer, P.; Cesa-Bianchi, N.; Fischer, P. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine Learning* **2002**, *47*, 235–256. <https://doi.org/10.1023/A:1013689704352>.
9. Russo, D.; Roy, B.V.; Kazerouni, A.; Osband, I.; Wen, Z. A Tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning* **2018**, *11*, 1–96. <https://doi.org/10.1561/22000000070>.
10. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-Level Control through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. <https://doi.org/10.1038/nature14236>.
11. Durrant-Whyte, H.; Bailey, T. Simultaneous Localization and Mapping: Part I. *IEEE Robotics & Automation Magazine* **2006**, *13*, 99–110. <https://doi.org/10.1109/MRA.2006.1638022>.
12. Cadena, C.; Carlone, L.; Carrillo, H.; Latif, Y.; Scaramuzza, D.; Neira, J.; Reid, I.; Leonard, J.J. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics* **2016**, *32*, 1309–1332. <https://doi.org/10.1109/TRO.2016.2624754>.
13. Bewley, A.; Ge, Z.; Ott, L.; Ramos, F.; Upcroft, B. Simple Online and Realtime Tracking. In Proceedings of the 2016 IEEE International Conference on Image Processing (ICIP), 2016, pp. 3464–3468. <https://doi.org/10.1109/ICIP.2016.7533003>.

14. Zhang, Y.; Sun, P.; Jiang, Y.; Yu, D.; Weng, F.; Yuan, Z.; Luo, P.; Liu, W.; Wang, X. ByteTrack: Multi-object Tracking by Associating Every Detection Box. In *Computer Vision – ECCV 2022*; Springer: Cham, 2022; pp. 1–21. https://doi.org/10.1007/978-3-031-20047-2_1.
1021
1022
1023
15. Fraunhofer IMS and AlfES Contributors. AlfES: Artificial Intelligence for Embedded Systems, 2024. Accessed 2026-04-15.
1024
1025

Appendix A Appendix: `sentai.io`

The `sentai.io` namespace groups the simplest board-local control primitives. In the current runtime it is intentionally small and is used mainly for visual status indication, quick feedback during debugging, and minimal interaction from scripts that do not require a richer GPIO abstraction.

Appendix A.1 Functions

- `sentai.io.led_on()`: Turns the user LED on.
- `sentai.io.led_off()`: Turns the user LED off.

Appendix A.2 Example

```
import sentai

for _ in range(3):
    sentai.io.led_on()
    sentai.rtos.sleep_ms(200)
    sentai.io.led_off()
    sentai.rtos.sleep_ms(200)
```

Appendix B Appendix: `sentai.rtos`

The `sentai.rtos` namespace exposes runtime observability and timing services from the FreeRTOS layer. It is useful both for interactive diagnosis and for scripts that need coarse scheduling, uptime measurement, or visibility into task and memory behavior during long-running onboard experiments.

Appendix B.1 Functions

- `sentai.rtos.sleep_ms(ms)`: Suspends the current script for a specified number of milliseconds.
- `sentai.rtos.ticks_ms()`: Returns a monotonic millisecond counter.
- `sentai.rtos.tasks()`: Returns the current task list with state, priority, and stack high-water mark.
- `sentai.rtos.heap_info()`: Returns memory statistics for the RTOS heap and MicroPython GC heap.
- `sentai.rtos.cpu_usage()`: Returns per-task CPU-usage estimates.
- `sentai.rtos.uptime()`: Returns system uptime in seconds.

Appendix B.2 Example

```
import sentai

print('uptime:', sentai.rtos.uptime())
print('heap:', sentai.rtos.heap_info())
for name, state, prio, hwm in sentai.rtos.tasks():
    print(name, state, prio, hwm)
```

Appendix C Appendix: `sentai.tpu`

The `sentai.tpu` namespace is the main interface to EdgeTPU-backed inference. It covers model loading, input preparation, execution, tensor inspection, quantization helpers, and detector-oriented post-processing. In practice it is the default path for compact vision models compiled for the accelerator.

Appendix C.1 Functions

- `sentai.tpu.load(path)`: Loads a `.tflite` model from flash and creates the EdgeTPU interpreter. 1069
- `sentai.tpu.load_image(path)`: Loads a JPEG or raw RGB image into the input tensor. 1070
- `sentai.tpu.invoke()`: Runs inference and returns elapsed time in milliseconds. 1071
- `sentai.tpu.ready()`: Reports whether the model and interpreter are ready. 1072
- `sentai.tpu.num_outputs()`: Returns the number of output tensors. 1073
- `sentai.tpu.output_size(idx)`: Returns the byte size of an output tensor. 1074
- `sentai.tpu.output(idx)`: Returns raw output bytes. 1075
- `sentai.tpu.output_dims(idx)`: Returns the output tensor shape. 1076
- `sentai.tpu.output_type(idx)`: Returns the tensor type. 1077
- `sentai.tpu.row(idx, r)`: Returns a selected tensor row. 1078
- `sentai.tpu.value(idx, flat_i)`: Returns a single flattened value. 1079
- `sentai.tpu.save_output(path)`: Saves all outputs to CSV. 1080
- `sentai.tpu.input_quant()`: Returns input quantization parameters. 1081
- `sentai.tpu.output_quant(idx)`: Returns output quantization parameters. 1082
- `sentai.tpu.output_floats(idx)`: Returns dequantized outputs as Python floats. 1083
- `sentai.tpu.input_type()`: Returns the input tensor type. 1084
- `sentai.tpu.detect(conf, iou, max)`: Runs YOLO-style non-maximum suppression on the last inference output. 1085
- `sentai.tpu.draw(path, dets, quality)`: Draws detections on the last captured frame and saves a JPEG. 1086

Appendix C.2 Example

```
import sentai
1092
1093
1094
sentai.tpu.load('/models/yolov8n.tflite')
1095
sentai.camera.init()
1096
sentai.camera.to_tensor()
1097
ms = sentai.tpu.invoke()
1098
dets = sentai.tpu.detect(0.25, 0.45, 50)
1099
print('ms:', ms, 'detections:', len(dets))
1100
sentai.tpu.draw('/det.jpg', dets)
1101
```

Appendix D Appendix: sentai.tfl

The `sentai.tfl` namespace provides CPU-side TensorFlow Lite Micro inference on the Cortex-M7. It complements `sentai.tpu` for models that are not compiled for the EdgeTPU, for smaller auxiliary models, or for experiments that require direct MCU execution with explicit tensor-arena sizing.

Appendix D.1 Functions

- `sentai.tfl.load(path, arena_kb)`: Loads a `.tflite` model and allocates the tensor arena. 1107
- `sentai.tfl.unload()`: Frees the interpreter and tensor arena. 1108
- `sentai.tfl.invoke()`: Runs inference and returns elapsed time in milliseconds. 1109
- `sentai.tfl.ready()`: Reports whether the interpreter is ready. 1110
- `sentai.tfl.info()`: Prints model and arena information. 1111
- `sentai.tfl.set_input(data)`: Writes bytes into the input tensor. 1112

- `sentai.tfl.load_image(path)`: Loads a JPEG or raw RGB image into the input tensor. 1115
- `sentai.tfl.input_size()`: Returns the input tensor size in bytes. 1116
- `sentai.tfl.input_dims()`: Returns the input tensor shape. 1117
- `sentai.tfl.input_type()`: Returns the input tensor type. 1119
- `sentai.tfl.input_quant()`: Returns input quantization parameters. 1120
- `sentai.tfl.num_outputs()`: Returns the number of output tensors. 1121
- `sentai.tfl.output_size(idx)`: Returns output tensor size. 1122
- `sentai.tfl.output(idx)`: Returns raw output bytes. 1123
- `sentai.tfl.output_dims(idx)`: Returns the output tensor shape. 1124
- `sentai.tfl.output_type(idx)`: Returns the output tensor type. 1125
- `sentai.tfl.output_quant(idx)`: Returns output quantization parameters. 1126
- `sentai.tfl.output_floats(idx)`: Returns dequantized output values. 1127
- `sentai.tfl.row(idx, r)`: Returns one tensor row. 1128
- `sentai.tfl.value(idx, flat_i)`: Returns one output value. 1129
- `sentai.tfl.save_output(path)`: Saves all outputs to CSV. 1130

Appendix D.2 Example

```
import sentai
```

```
sentai.tfl.load('/models/keyword_detect.tflite', 256)
sentai.tfl.set_input(feature_bytes)
ms = sentai.tfl.invoke()
probs = sentai.tfl.output_floats(0)
print('inference:', ms, 'ms', probs)
```

Appendix E Appendix: sentai.fs

The `sentai.fs` namespace provides script-level access to the LittleFS user partition. It is used to move models, scripts, logs, images, and outputs between the runtime and persistent flash, and it is a central part of the interactive workflow because both model deployment and experiment logging depend on it.

Appendix E.1 Functions

- `sentai.fs.read(path)`: Reads an entire file as bytes. 1145
- `sentai.fs.read_str(path)`: Reads an entire file as text. 1146
- `sentai.fs.read_base64(path)`: Reads and prints a file as base64 text. 1147
- `sentai.fs.write(path, data)`: Writes bytes or text to flash. 1148
- `sentai.fs.size(path)`: Returns the file size. 1149
- `sentai.fs.exists(path)`: Checks whether a file or directory exists. 1150
- `sentai.fs.remove(path)`: Deletes a file or empty directory. 1151
- `sentai.fs.mkdir(path)`: Creates directories recursively. 1152
- `sentai.fs.ls(path)`: Lists directory contents. 1153
- `sentai.fs.format()`: Reformats the user partition and erases user data. 1154

Appendix E.2 Example

```
import sentai
```

```
sentai.fs.mkdir('/logs')
sentai.fs.write('/logs/run.txt', 'experiment started\n')
print(sentai.fs.read_str('/logs/run.txt'))
```

```
print(sentai.fs.ls('/logs'))
```

Appendix F Appendix: sentai.camera

The `sentai.camera` namespace controls image capture and basic camera-side preprocessing. It provides the main bridge between the sensing layer and the inference pipeline, including frame capture, JPEG export, tensor feeding, camera selection, and frame-sequence inspection for dual-camera operation.

Appendix F.1 Functions

- `sentai.camera.init(streaming)`: Starts the camera in continuous or trigger mode.
- `sentai.camera.stop()`: Stops camera capture.
- `sentai.camera.jpeg(quality)`: Captures a frame and returns JPEG bytes.
- `sentai.camera.save_jpeg(path, q)`: Captures and saves a JPEG file.
- `sentai.camera.to_tensor()`: Captures a frame and writes it into the TPU input tensor.
- `sentai.camera.resolution()`: Returns the current output resolution.
- `sentai.camera.set_resolution(w, h)`: Changes the capture resolution.
- `sentai.camera.native_res()`: Returns the sensor native resolution.
- `sentai.camera.select(id)`: Selects the active camera.
- `sentai.camera.frame_count()`: Returns the monotonic hardware frame counter.

Appendix F.2 Example

```
import sentai

sentai.camera.init()
sentai.camera.set_resolution(320, 320)
sentai.camera.select(0)
sentai.camera.save_jpeg('/front.jpg', 85)
sentai.camera.select(1)
sentai.camera.save_jpeg('/back.jpg', 85)
sentai.camera.stop()
```

Appendix G Appendix: sentai.imu

The `sentai.imu` namespace exposes the onboard LIS2DU12 accelerometer. It provides direct access to acceleration and derived tilt estimates and is commonly used for motion monitoring, camera-pose estimation, and simple event detection in scripts that combine perception with inertial context.

Appendix G.1 Functions

- `sentai.imu.init()`: Initializes the accelerometer.
- `sentai.imu.read()`: Returns acceleration and temperature measurements.
- `sentai.imu.degrees()`: Returns pitch and roll in degrees.
- `sentai.imu.radians()`: Returns pitch and roll in radians.

Appendix G.2 Example

```
import sentai

sentai.imu.init()
for _ in range(10):
    d = sentai.imu.degrees()
```

```

if d:
    print('pitch:', d['pitch'], 'roll:', d['roll'])
sentai.rtos.sleep_ms(100)

```

Appendix H Appendix: sentai.mic

The `sentai.mic` namespace manages the onboard PDM microphone through a ring-buffer recording model. It is designed for low-overhead acquisition, sound-level monitoring, and asynchronous MP3 export, allowing audio-triggered experiments without forcing the user to manage raw DMA buffers directly.

Appendix H.1 Functions

- `sentai.mic.start(seconds)`: Starts ring-buffer recording for the specified window length.
- `sentai.mic.stop()`: Stops the microphone and returns the number of buffered samples.
- `sentai.mic.recording()`: Reports whether the microphone is active.
- `sentai.mic.samples()`: Returns the number of samples currently available.
- `sentai.mic.level()`: Returns the current RMS level in centi-decibels.
- `sentai.mic.save_mp3()`: Saves buffered audio as an MP3 file and returns the filename.

Appendix H.2 Example

```

import sentai

sentai.mic.start(5)
sentai.rtos.sleep_ms(5000)
name = sentai.mic.save_mp3()
print('saved:', name)
sentai.mic.stop()

```

Appendix I Appendix: sentai.usb

The `sentai.usb` namespace provides two distinct services: mass-storage export of the user partition and raw USB CDC serial I/O when the REPL has been moved away from USB. It is therefore both a deployment path for files and a host-device communication channel, with explicit coordination required to avoid conflicts with filesystem access.

Appendix I.1 Functions

- `sentai.usb.drive(on)`: Enables or disables USB mass-storage export.
- `sentai.usb.open()`: Opens USB CDC ACM for raw serial I/O.
- `sentai.usb.close()`: Closes the raw USB CDC channel.
- `sentai.usb.write(data)`: Writes bytes or text to the USB host.
- `sentai.usb.read(max, timeout_ms)`: Reads bytes from the USB host.
- `sentai.usb.available()`: Returns the number of pending received bytes.

Appendix I.2 Example

```

import sentai

sentai.usb.drive(1)
# host copies files here
sentai.usb.drive(0)

```

```
print(sentai.fs.ls('/'))
```

1249

Appendix J Appendix: sentai.uart

1250

The `sentai.uart` namespace exposes raw UART serial I/O on the board-side serial port when that port is not reserved for the REPL or another protocol bridge. It is intended for direct communication with external serial devices such as radios, sensors, GPS modules, or custom peripherals.

1251

1252

1253

1254

Appendix J.1 Functions

1255

- `sentai.uart.open(baud)`: Opens the UART port at the requested baud rate.
- `sentai.uart.close()`: Closes the UART port and restores the default configuration.
- `sentai.uart.write(data)`: Sends bytes or text.
- `sentai.uart.read(max, timeout_ms)`: Reads bytes with polling, timeout, or blocking behavior.
- `sentai.uart.available()`: Returns the number of waiting bytes.

1256

1257

1258

1259

1260

1261

Appendix J.2 Example

1262

```
import sentai
```

1263

1264

```
sentai.uart.open(38400)
```

1265

```
sentai.uart.write(b'AT\r\n')
```

1266

```
resp = sentai.uart.read(256, 1000)
```

1267

```
print(resp)
```

1268

```
sentai.uart.close()
```

1269

Appendix K Appendix: sentai.console

1270

The `sentai.console` interface controls where the interactive MicroPython REPL is exposed. Although small, it is operationally important because several communication namespaces reuse the non-REPL interface, so moving the console between USB and UART determines which transport remains available for scripting.

1271

1272

1273

1274

Appendix K.1 Functions

1275

- `sentai.console()`: Returns the current REPL target, either `usb` or `uart`.
- `sentai.console('usb')`: Moves the REPL to USB CDC ACM.
- `sentai.console('uart')`: Moves the REPL to the UART console.

1276

1277

1278

Appendix K.2 Example

1279

```
import sentai
```

1280

1281

```
print(sentai.console())
```

1282

```
sentai.console('uart')
```

1283

```
sentai.usb.open()
```

1284

```
sentai.usb.write(b'host channel ready\n')
```

1285

```
sentai.usb.close()
```

1286

Appendix L Appendix: sentai.mesh

1287

The `sentai.mesh` namespace bridges the runtime to Meshtastic-compatible radios over UART. It supports both simple text exchange and structured vision-message transport, making it suitable for distributed sensing, low-bandwidth telemetry, and multi-node

1288

1289

1290

experiments in which detections or updates must be serialized and disseminated over a mesh network.

Appendix L.1 Functions

- `sentai.mesh.init(baud)`: Opens the UART link and starts the mesh receive task.
- `sentai.mesh.stop()`: Stops the receive task and closes the link.
- `sentai.mesh.node()`: Returns the local node number.
- `sentai.mesh.config()`: Requests configuration and NodeDB information from the radio.
- `sentai.mesh.send(text, dest, ch, ack)`: Sends a text message.
- `sentai.mesh.available()`: Returns the number of queued text messages.
- `sentai.mesh.receive(timeout)`: Receives the next text message.
- `sentai.mesh.send_detection(...)`: Sends a structured new-detection message.
- `sentai.mesh.send_update(...)`: Sends a structured track-update message.
- `sentai.mesh.vision_available()`: Returns the number of queued vision messages.
- `sentai.mesh.receive_vision(timeout)`: Receives the next structured vision message.
- `sentai.mesh.set_pose(pitch_deg, roll_deg, altitude_cm, heading_deg)`: Attaches camera pose metadata to outgoing vision messages.

Appendix L.2 Example

```
import sentai

sentai.mesh.init()
sentai.mesh.send('hello world')
msg = sentai.mesh.receive(5000)
if msg:
    print(msg['from'], msg['text'])
```

Appendix M Appendix: `sentai.link`

The `sentai.link` namespace exposes a MAVLink v2 bridge over UART. It supports message reception, heartbeat and status transmission, command dispatch, structured vision reporting, and obstacle-map generation for PX4 collision prevention. It is the primary interface for integrating the runtime with a conventional autopilot stack.

Appendix M.1 Functions

- `sentai.link.init(baud, sysid, compid)`: Opens the MAVLink link and starts the receive task.
- `sentai.link.stop()`: Stops the link and closes the UART channel.
- `sentai.link.available()`: Returns the number of queued MAVLink messages.
- `sentai.link.receive(timeout)`: Returns the next decoded MAVLink message.
- `sentai.link.heartbeat(type)`: Sends a heartbeat.
- `sentai.link.send(text, sev)`: Sends a STATUSTEXT message.
- `sentai.link.command(tsys, tcomp, cmd, conf, p1..p7)`: Sends a COMMAND_LONG message.
- `sentai.link.send_detection(...)`: Sends a structured vision detection.
- `sentai.link.send_update(...)`: Sends a structured vision update.
- `sentai.link.send_delete(...)`: Sends a structured vision deletion event.
- `sentai.link.obstacles_from_tracker(...)`: Builds and sends a 72-bin obstacle map from active tracks.

- `sentai.link.obstacle_distance(distances72, ...)`: Sends a raw `OBSTACLE_DISTANCE` message. 1338
- `sentai.link.obstacles_from_points(points, ...)`: Builds an obstacle map from arbitrary XY points. 1339
1340

Appendix M.2 Example

```
import sentai
1341
1342
1343
sentai.pipeline.init()
1344
sentai.pipeline.set_pose(120, -1, 0.0, 0.0)
1345
sentai.link.init(57600)
1346
while True:
1347
    sentai.link.obstacles_from_tracker(800, 20)
1348
    sentai.link.heartbeat()
1349
    sentai.rtos.sleep_ms(100)
1350
```

Appendix N Appendix: `sentai.crazy`

The `sentai.crazy` namespace connects the runtime to a Crazyflie platform over CPX/CRTP. It exposes both higher-level flight procedures and lower-level control interfaces, allowing scripted indoor flight experiments without moving mission logic outside the main MicroPython environment.

Appendix N.1 Functions

- `sentai.crazy.init(baud)`: Starts the Crazyflie communication tasks. 1351
1352
- `sentai.crazy.stop()`: Stops the bridge and closes the link. 1353
- `sentai.crazy.debug(level)`: Changes debug verbosity. 1354
- `sentai.crazy.arm()`: Arms the drone. 1355
- `sentai.crazy.disarm()`: Disarms the drone. 1356
- `sentai.crazy.ping(timeout)`: Measures round-trip time to the drone. 1357
- `sentai.crazy.fly(height, hold_ms, takeoff_ms, land_ms)`: Executes a blocking high-level flight cycle. 1358
1359
- `sentai.crazy.attitude(roll, pitch, yaw_rate, thrust)`: Sends non-blocking attitude setpoints. 1360
1361
- `sentai.crazy.fly_stop()`: Stops attitude-controlled flight and disarms. 1362
- `sentai.crazy.takeoff(h, dur, yaw, use_yaw, group)`: Sends a high-level takeoff command. 1363
1364
- `sentai.crazy.land(h, dur, yaw, use_yaw, group)`: Sends a high-level landing command. 1365
1366
- `sentai.crazy.stop_motors(group)`: Performs an emergency motor stop. 1367
- `sentai.crazy.go_to(x, y, z, yaw, dur, rel, lin, group)`: Sends a waypoint-style movement command. 1368
1369
- `sentai.crazy.hover(vx, vy, yr, z)`: Sends hover-mode velocity commands. 1370
- `sentai.crazy.test_fly(power, dur_ms)`: Runs a raw motor test. 1371
- `sentai.crazy.send_crtp(port, ch, data)`: Sends an arbitrary CRTP packet. 1372
1373
1374
1375
1376
1377

Appendix N.2 Example

```
import sentai
1378
1379
1380
sentai.crazy.init()
1381
sentai.crazy.arm()
1382
```

```

sentai.crazy.takeoff(0.5, 2.0)
sentai.rtos.sleep_ms(3000)
sentai.crazy.land(0.0, 2.0)
sentai.crazy.stop()

```

Appendix O Appendix: sentai.pipeline

The `sentai.pipeline` namespace exposes the background detection pipeline and the SentAI-SORT tracker. It is the main high-level perception service in the runtime and packages detector scheduling, tracking, event reporting, camera geometry, and ground-plane projection into a single script-facing interface.

Appendix O.1 Functions

- `sentai.pipeline.start(conf, iou, max, track)`: Starts continuous detection and optionally tracking.
- `sentai.pipeline.stop()`: Stops the pipeline.
- `sentai.pipeline.running()`: Reports whether the pipeline is active.
- `sentai.pipeline.get(timeout_ms)`: Returns the next detection frame.
- `sentai.pipeline.stats()`: Returns processed, dropped, and FPS statistics.
- `sentai.pipeline.tracks()`: Returns the current active-track snapshot.
- `sentai.pipeline.event(timeout_ms)`: Returns the next tracker event.
- `sentai.pipeline.track_config(...)`: Gets or sets tracker parameters.
- `sentai.pipeline.set_pose(altitude_cm, heading_deg, lat, lon)`: Sets camera pose for ground projection.
- `sentai.pipeline.camera_config(cam_id, fov_h, fov_v, mount_pitch, mount_roll, mount_yaw)`: Gets or sets per-camera geometry.

Appendix O.2 Example

```

import sentai

sentai.camera.init()
sentai.pipeline.camera_config(0, 70.8, 43.4, 0, 0, 0)
sentai.pipeline.set_pose(1000, 0, 44.4268, 26.1025)
sentai.pipeline.start(0.3, 0.45, 50, True)
while True:
    evt = sentai.pipeline.event(2000)
    if evt:
        print(evt)

```

Appendix P Appendix: sentai.aifes

The `sentai.aifes` namespace integrates the AIFES library for on-device neural-network training and inference. It is intended for small multilayer perceptrons and compact transfer-learning workflows in which features extracted by the TPU or camera are fed into a trainable MCU-side head.

Appendix P.1 Functions

- `sentai.aifes.load(path)`: Loads a model architecture from YAML.
- `sentai.aifes.load_weights(path)`: Loads pre-trained weights.
- `sentai.aifes.save_weights(path)`: Saves trained weights.
- `sentai.aifes.unload()`: Frees the loaded model.
- `sentai.aifes.ready()`: Reports whether the model is ready.

- `sentai.aifes.train(x_data, y_data, ...)`: Trains the model on supplied samples. 1428
- `sentai.aifes.set_input(data)`: Sets the input vector from floats. 1429
- `sentai.aifes.from_tpu(idx)`: Uses a TPU output tensor as AIFES input. 1430
- `sentai.aifes.from_camera(w, h, gray)`: Uses a camera frame as normalized input. 1431
- `sentai.aifes.from_mic(samples)`: Uses microphone samples as input. 1432
- `sentai.aifes.invoke()`: Runs inference. 1433
- `sentai.aifes.output()`: Returns the latest output vector. 1434
- `sentai.aifes.mic_init()`: Initializes microphone capture for audio models. 1435
- `sentai.aifes.mic_stop()`: Stops microphone capture. 1436
- `sentai.aifes.predict(input)`: Legacy single-call inference helper. 1437
- `sentai.aifes.info()`: Returns model metadata. 1438

Appendix P.2 Example

```
import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.aifes.load('/models/classifier_head.yaml')
sentai.camera.init()

sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.aifes.from_tpu(0)
sentai.aifes.invoke()
print(sentai.aifes.output())
```

Appendix Q Appendix: `sentai.kmeans`

The `sentai.kmeans` namespace implements compact k-means clustering for embedding vectors. It is mainly useful for few-shot or unsupervised classification workflows in which a TPU feature extractor produces dense embeddings and the runtime clusters them directly on device.

Appendix Q.1 Functions

- `sentai.kmeans.init(k, dim)`: Initializes the clustering state. 1453
- `sentai.kmeans.add(cluster_idx, vector)`: Adds a labeled vector to a cluster. 1454
- `sentai.kmeans.add_from_tpu(cluster_idx, tpu_idx)`: Adds a TPU output directly. 1455
- `sentai.kmeans.compute()`: Computes centroids from labeled samples. 1456
- `sentai.kmeans.fit(max_iter)`: Runs unsupervised Lloyd iterations. 1457
- `sentai.kmeans.predict(vector)`: Predicts the closest centroid. 1458
- `sentai.kmeans.from_tpu(tpu_idx)`: Predicts from a TPU output tensor. 1459
- `sentai.kmeans.distances(vector)`: Returns distances to all centroids. 1460
- `sentai.kmeans.save(path)`: Saves centroids to flash. 1461
- `sentai.kmeans.load(path)`: Loads centroids from flash. 1462
- `sentai.kmeans.centroid(idx)`: Returns one centroid vector. 1463
- `sentai.kmeans.info()`: Returns configuration and sample-count metadata. 1464
- `sentai.kmeans.clear()`: Clears training vectors while keeping centroids. 1465

Appendix Q.2 Example

```

import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.camera.init()
sentai.kmeans.init(3, 1280)

sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.kmeans.add_from_tpu(0, 0)
sentai.kmeans.compute()
print(sentai.kmeans.info())

```

Appendix R Appendix: sentai.pca

The `sentai.pca` namespace provides principal-component analysis for high-dimensional embeddings. Its role is to reduce vector dimension before downstream use in classifiers, clustering methods, or lightweight anomaly detectors, especially when raw feature vectors are too large for repeated on-device processing.

Appendix R.1 Functions

- `sentai.pca.init(in_dim, out_dim)`: Initializes the PCA model.
- `sentai.pca.add(vector)`: Adds a training vector.
- `sentai.pca.from_tpu(idx)`: Adds a TPU output tensor as training data.
- `sentai.pca.fit()`: Computes the principal components.
- `sentai.pca.transform(vector)`: Projects a vector into the reduced space.
- `sentai.pca.transform_tpu(idx)`: Projects a TPU output tensor directly.
- `sentai.pca.inverse(reduced)`: Reconstructs an approximate original vector.
- `sentai.pca.explained_variance()`: Returns component variances.
- `sentai.pca.save(path)`: Saves the fitted PCA model.
- `sentai.pca.load(path)`: Loads a PCA model.
- `sentai.pca.info()`: Returns model metadata.
- `sentai.pca.clear()`: Clears stored training vectors.

Appendix R.2 Example

```

import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.camera.init()
sentai.pca.init(1280, 32)
sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.pca.from_tpu(0)
sentai.pca.fit()
print(sentai.pca.transform_tpu(0)[:5])

```

Appendix S Appendix: sentai.anomaly

The `sentai.anomaly` namespace implements online anomaly detection based on incremental covariance estimation and Mahalanobis distance, together with a simple CUSUM change detector. It is suitable for learning a baseline normal distribution and then scoring embeddings or scalar signals during deployment.

Appendix S.1 Functions

- `sentai.anomaly.init(dim)`: Initializes the anomaly model. 1519
- `sentai.anomaly.observe(vector)`: Updates the normal model with one observation. 1520
- `sentai.anomaly.observe_tpu(idx)`: Updates the model from a TPU output tensor. 1522
- `sentai.anomaly.score(vector)`: Returns the anomaly score for a vector. 1523
- `sentai.anomaly.score_tpu(idx)`: Scores a TPU output tensor. 1524
- `sentai.anomaly.threshold(val)`: Gets or sets the anomaly threshold. 1525
- `sentai.anomaly.is_anomaly(vector)`: Tests whether a vector exceeds the threshold. 1526
- `sentai.anomaly.is_anomaly_tpu(idx)`: Tests a TPU output tensor. 1527
- `sentai.anomaly.cusum_init(threshold, drift)`: Initializes the CUSUM detector. 1528
- `sentai.anomaly.cusum_observe(value)`: Feeds one scalar sample into CUSUM. 1529
- `sentai.anomaly.cusum_score()`: Returns current positive and negative cumulative sums. 1530
- `sentai.anomaly.cusum_reset()`: Clears CUSUM accumulators. 1532
- `sentai.anomaly.save(path)`: Saves the learned model. 1533
- `sentai.anomaly.load(path)`: Loads a saved model. 1534
- `sentai.anomaly.stats()`: Returns model statistics. 1535
- `sentai.anomaly.info()`: Alias for `stats()`. 1536
- `sentai.anomaly.clear()`: Clears all statistics. 1537

Appendix S.2 Example

```

import sentai
1538
1539
1540
sentai.tpu.load('/models/mobilenet_features.tflite')
1541
sentai.camera.init()
1542
sentai.anomaly.init(1280)
1543
1544
for _ in range(30):
1545
    sentai.camera.to_tensor()
1546
    sentai.tpu.invoke()
1547
    sentai.anomaly.observe_tpu(0)
1548
1549
sentai.camera.to_tensor()
1550
sentai.tpu.invoke()
1551
print(sentai.anomaly.score_tpu(0))
1552

```

Appendix T Appendix: sentai.dtw

The `sentai.dtw` namespace exposes dynamic time warping for template-based sequence matching. It is intended for short gesture or audio patterns where full trainable sequence models are unnecessary and a small number of stored templates is sufficient for online recognition.

Appendix T.1 Functions

- `sentai.dtw.init(dim, max_len)`: Initializes the DTW engine. 1559
- `sentai.dtw.record_start(label)`: Starts recording a named template. 1560
- `sentai.dtw.record_add(frame)`: Adds one feature frame. 1561
- `sentai.dtw.record_add_imu()`: Adds the current IMU sample as a frame. 1562
- `sentai.dtw.record_add_mic()`: Adds the current microphone level as a frame. 1563
- `sentai.dtw.record_end()`: Finalizes and stores the template. 1564

- `sentai.dtw.match(sequence, threshold)`: Matches a supplied sequence against stored templates. 1565
- `sentai.dtw.match_imu(n_frames, threshold, delay_ms)`: Captures IMU frames and matches them. 1566
- `sentai.dtw.match_mic(n_frames, threshold, delay_ms)`: Captures microphone frames and matches them. 1567
- `sentai.dtw.templates()`: Lists stored templates. 1568
- `sentai.dtw.remove(label)`: Removes a template. 1569
- `sentai.dtw.save(path)`: Saves all templates. 1570
- `sentai.dtw.load(path)`: Loads saved templates. 1571
- `sentai.dtw.info()`: Returns engine metadata. 1572
- `sentai.dtw.clear()`: Clears all templates. 1573

Appendix T.2 Example 1574

```
import sentai 1575
1576
1577
sentai.imu.init() 1578
sentai.dtw.init(3, 100) 1579
sentai.dtw.record_start('wave') 1580
for _ in range(50): 1581
    sentai.dtw.record_add_imu() 1582
    sentai.rtos.sleep_ms(20) 1583
sentai.dtw.record_end() 1584
print(sentai.dtw.match_imu(50, 500.0)) 1585
```

Appendix U Appendix: sentai.hmm 1586

The `sentai.hmm` namespace implements a discrete hidden Markov model with Baum-Welch training and Viterbi decoding. It is suitable for compact sequence-classification tasks in which observations can be discretized, such as simple activity recognition or discretized feature streams extracted from sensors. 1587

Appendix U.1 Functions 1588

- `sentai.hmm.init(n_states, n_obs)`: Initializes the HMM. 1589
- `sentai.hmm.set_transition(i, j, prob)`: Sets a transition probability. 1590
- `sentai.hmm.set_emission(state, obs, prob)`: Sets an emission probability. 1591
- `sentai.hmm.set_prior(state, prob)`: Sets an initial-state probability. 1592
- `sentai.hmm.add_seq(obs_list)`: Adds a training sequence. 1593
- `sentai.hmm.train(max_iter)`: Runs Baum-Welch training. 1594
- `sentai.hmm.viterbi(obs_list)`: Decodes the most likely state sequence. 1595
- `sentai.hmm.predict(obs)`: Predicts the most likely next state. 1596
- `sentai.hmm.log_likelihood(obs_list)`: Scores a sequence. 1597
- `sentai.hmm.from_tpu(idx, n_bins)`: Discretizes a TPU output tensor. 1598
- `sentai.hmm.from_imu(n_bins)`: Discretizes IMU magnitude. 1599
- `sentai.hmm.save(path)`: Saves the HMM model. 1600
- `sentai.hmm.load(path)`: Loads a saved model. 1601
- `sentai.hmm.info()`: Returns model metadata. 1602
- `sentai.hmm.clear()`: Clears parameters and sequences. 1603

Appendix U.2 Example 1604

```
import sentai 1605
```

```

sentai.imu.init()
sentai.hmm.init(3, 8)
seq = [sentai.hmm.from_imu(8) for _ in range(100)]
sentai.hmm.add_seq(seq)
sentai.hmm.train(20)
print(sentai.hmm.predict(sentai.hmm.from_imu(8)))

```

Appendix V Appendix: sentai.rl

The `sentai.rl` namespace collects three reinforcement-learning mechanisms with different complexity levels: tabular Q-learning, multi-armed bandits, and a compact DQN. It is intended for online adaptation of thresholds, policies, or configuration choices in small embedded experiments.

Appendix V.1 Functions

- `sentai.rl.q_init(n_states, n_actions)`: Initializes a tabular Q-table.
- `sentai.rl.q_update(s, a, r, s_next, alpha, gamma)`: Updates one Q-value.
- `sentai.rl.q_action(s, epsilon)`: Selects an action with an epsilon-greedy policy.
- `sentai.rl.q_value(s, a)`: Returns one Q-value.
- `sentai.rl.q_save(path)`: Saves the Q-table.
- `sentai.rl.q_load(path)`: Loads the Q-table.
- `sentai.rl.q_clear()`: Clears the Q-table.
- `sentai.rl.mab_init(n_arms)`: Initializes a multi-armed bandit.
- `sentai.rl.mab_pull(arm, reward)`: Reports the reward of a selected arm.
- `sentai.rl.mab_select(strategy)`: Selects the next arm.
- `sentai.rl.mab_stats()`: Returns per-arm statistics.
- `sentai.rl.mab_clear()`: Clears bandit statistics.
- `sentai.rl.dqn_init(state_dim, n_actions, hidden)`: Initializes the DQN.
- `sentai.rl.dqn_observe(state, action, reward, next_state, done)`: Adds one replay transition.
- `sentai.rl.dqn_action(state, epsilon)`: Selects an action from the DQN.
- `sentai.rl.dqn_train(batch_size)`: Trains on a replay batch.
- `sentai.rl.dqn_save(path)`: Saves DQN weights.
- `sentai.rl.dqn_load(path)`: Loads DQN weights.
- `sentai.rl.dqn_clear()`: Clears DQN state.
- `sentai.rl.info()`: Returns a summary of all three subsystems.

Appendix V.2 Example

```

import sentai

sentai.rl.mab_init(4)
for _ in range(20):
    arm = sentai.rl.mab_select(1)
    reward = 1.0 if arm == 0 else 0.2
    sentai.rl.mab_pull(arm, reward)
print(sentai.rl.mab_stats())

```

Appendix W Appendix: sentai.slam

The `sentai.slam` namespace implements a compact detection-based EKF-SLAM system. It estimates a planar robot pose and maintains a landmark map using object detections

or manually supplied observations. The interface is decoupled from the TPU, so it can consume detections from any compatible source.

Appendix W.1 Functions

- `sentai.slam.init(fov_h_deg, img_w, img_h, max_lm, baseline_m)`: Initializes the SLAM model.
- `sentai.slam.update(detections)`: Updates pose and landmarks from monocular detections.
- `sentai.slam.update_stereo(dets_left, dets_right)`: Updates from stereo detections.
- `sentai.slam.observe(class_id, bearing_rad, range_m)`: Adds a manual landmark observation.
- `sentai.slam.predict(dx, dy, dtheta)`: Applies an explicit motion prediction.
- `sentai.slam.pose()`: Returns the current estimated pose.
- `sentai.slam.landmarks()`: Returns the active landmark set.
- `sentai.slam.noise(sigma_v, sigma_w, sigma_b, sigma_r)`: Gets or sets noise parameters.
- `sentai.slam.imu_correct(pitch_deg, roll_deg)`: Adjusts bearing noise using IMU tilt.
- `sentai.slam.clear()`: Resets pose and landmarks.
- `sentai.slam.save(path)`: Saves the map and pose.
- `sentai.slam.load(path)`: Loads the map and pose.
- `sentai.slam.info()`: Returns SLAM metadata and statistics.

Appendix W.2 Example

```
import sentai

sentai.slam.init(66.0, 320, 320)
sentai.tpu.load('/models/yolov8n_320.tflite')
sentai.camera.init()

sentai.camera.to_tensor()
sentai.tpu.invoke()
dets = sentai.tpu.detect(0.3, 0.45, 50)
sentai.slam.update(dets)
print(sentai.slam.pose(), sentai.slam.landmarks())
```